SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY – INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Robust Optimization of Charging Station Placement for Electric Trucks in Grocery Retail

Max Elia Schweigkofler

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY – INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Robust Optimization of Charging Station Placement for Electric Trucks in Grocery Retail

Robuste Optimierung der Ladesäulenplatzierung für Elektro-Lkws im Lebensmitteleinzelhandel

Author:	Max Elia Schweigkofler
Examiner:	Prof. Dr. Clemens Thielen
Supervisor:	Johannes R. Kager, M.Sc.
Submission Date:	04.05.2026

I confirm that this master's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 04.05.2026 Max Elia Schweigkofler

Abstract

A retailer that wants to replace diesel trucks with battery-electric trucks faces a planning problem with long-term decisions and uncertain daily demand. Installing chargers at stores is expensive and hard to reverse once the chargers are in place. Demand, however, changes from day to day, so a plan that works for an average day may fail on a difficult day. That is why this thesis tackles the question of where to install chargers as a robust optimization problem which optimizes for the worst case scenario out of a given set of demand scenarios. This approach is valid since the infrastructure decision is irreversible and has to perform reasonably well on every day and have no very expensive outliers.

The data basis for this problem comes from real delivery records from a large German grocery retailer which is transformed into optimizable instances with daily demand, distance matrices, and empirical unloading time estimates. The charging station placement problem is then formulated mathematically as a two-stage robust mixed integer program which is then implemented to solve efficiently with a recent column and constraint generation approach for finite scenario two-stage robust problems. In the first stage the solver needs to decide at which stores to install chargers and of what type. In the second stage the solver calculates the routing and the charging based on that infrastructure taking into account the maximum load capacity of the truck and the battery state of charge. Storing at individual stores has one particular advantage, the time the trucks stop to unload can be used to charge for free and enable them to ride longer tours. This decentralised charging architecture also minimizes the dependence on warehouse charging, where a large fleet of many trucks charging in parallel might possibly exceed what the local grid connection can provide. In order to find good values for the individual parameters the historical delivery data and technical truck specifications were analyzed. The vehicle class used primarily was the Mercedes-Benz eActros 300 with the Volvo FM Electric as a secondary option.

Trying to solve the model became computationally difficult quite fast, as the number of stores increased, which made it necessary to embed it in a pipeline which splits the problem into smaller instances. That is why clustering had to be treated as a core part of the algorithm. The implemented clustering family includes a simple geographic method, store-count angular slices, demand-aware angular slices, and a tour-containment variant that preserves whole historical tours as good as possible. The four methods go from simple to

more operations-aware: geographic clustering only looks at location; store-count angular slices cut the map into equal-size pie slices around the depot; demand-aware angular slices cut the same slices but balance demand instead of store count; and tour-containment tries to keep whole historical tours inside the same cluster. A comparison on a 96-store test instance shows that demand-aware angular slices achieve the lowest worst-case cost after reoptimization, while the store-count variant is about 16 % more expensive on the worst scenario. Demand-aware angular slices are used as the default method, because they give the lowest worst-case cost after reoptimization, do not rely on historical tour data, and always give the same result for the same input, which ensures reproducibility, unlike the geographic k-means method whose outcome depends on the random initialization. Additional runtime experiments show that good first-stage solutions often stabilize much earlier than the reported MIP gap suggests, which supports tighter bounded-time budgets for easier clustered subproblems and longer budgets for structurally harder ones. The final experiment uses the full available retailer data with 259 stores and 31 historical demand scenarios. Two different cluster sizes were run on this instance. The bigger one produced a significantly lower worst-case cost but needed much more computational time than the smaller one.

The final outcome of the thesis is the full robust CSPP pipeline that minimizes costs for long term charger placement decisions and short term routing. It evaluates if installing store-side chargers would eventually be cheaper overall than relying on warehouse charging only. The thesis also transitioned from only solving the pipeline for one specific retailer only, to creating a web-based tool that integrates that pipeline and allows other retailers to import their regular route stops and demand scenarios and optimize them with the same underlying model. A reoptimization study on the 12-stores-per-cluster variant and a cost-parameter sweep on a single 16-store cluster are also reported, showing how the placement decision reacts to changes in the underlying cost assumptions.

Contents

Abstract **iii**

1 Introduction **1**
1.1 The Case for Electrifying Grocery Delivery . . . 2
1.2 Research Questions . . . 2
1.3 Contributions . . . 3
1.4 Thesis Structure . . . 3

2 Problem Setting and Data Basis **4**
2.1 The Retailer Data . . . 4
2.2 Choosing Representative Electric Trucks . . . 4
2.3 Unloading Time . . . 5
2.4 Energy Consumption and State-of-Charge Assumptions . . . 6
2.5 Charging Infrastructure Assumptions . . . 6
2.5.1 Charger Types and Installation Costs . . . 6
2.6 Optimization Instances . . . 7

3 Related Work **8**
3.1 Electric Vehicle Routing and Store Charging . . . 8
3.2 Robust Optimization . . . 8
3.3 Clustering . . . 9
3.4 Positioning of This Thesis . . . 10

4 Charging Station Placement Problem **11**
4.1 Problem Setting and Worst-Case Objective . . . 11
4.2 Mathematical Formulation . . . 12
4.2.1 Problem Structure and Notation Mapping . . . 12
4.2.2 Sets and Parameters . . . 13
4.2.3 First-Stage Decision Variables . . . 14
4.2.4 Second-Stage Decision Variables . . . 14
4.2.5 Objective Function . . . 17
4.2.6 Constraints . . . 18

4.2.7 The Full CSPP . . . 23
4.3 Instance Preprocessing . . . 23
4.3.1 Split-Delivery Preprocessing . . . 23
4.3.2 Fleet Sizing and Multi-Tour Modeling . . . 23

5 Parameter Calibration **25**
5.1 Vehicle and Energy Parameters . . . 25
5.2 Cost Parameters . . . 26
5.3 Charging Infrastructure . . . 27

6 Solution Algorithm **29**
6.1 Model Complexity . . . 29
6.2 Clustering . . . 30
6.2.1 Geographic clustering . . . 30
6.2.2 Angular slice clustering . . . 31
6.2.3 Tour-containment clustering . . . 32
6.3 Stage 1: Scenario Addition with Bound-Based Pruning . . . 33
6.3.1 Algorithmic idea . . . 33
6.3.2 Example: Cluster 6 . . . 34
6.4 Stage 2: Scenario Evaluation . . . 35
6.5 Stage 3: Reoptimization . . . 35
6.5.1 Loop structure . . . 35
6.5.2 Reoptimization Behavior . . . 36
6.6 Pipeline Summary . . . 38

7 Runtime Calibration **39**
7.1 Stage 1 cluster size calibration . . . 39
7.1.1 Setup . . . 39
7.1.2 Results . . . 39
7.2 Stage 2 Plateau Experiment . . . 40
7.2.1 Setup . . . 40
7.2.2 Results . . . 40
7.3 Total Number of MIP Solves . . . 41
7.4 Parallel Execution . . . 42
7.5 Bucket Policy . . . 43
7.6 Time Limits in Action . . . 44

8 Results **45**
8.1 Clustering Method Comparison . . . 45
8.1.1 Setup . . . 45
8.1.2 Results . . . 45
8.1.3 Interpretation . . . 45
8.1.4 Choice for the Final Experiments . . . 47
8.2 Cost Sensitivity Study . . . 48
8.2.1 Setup . . . 48
8.2.2 Results . . . 48
8.3 Full-Instance Runs . . . 49
8.3.1 Data and Settings . . . 49
8.3.2 Scope and Consistency . . . 50
8.4 Full-Instance Results . . . 50
8.4.1 Store Layout and Demand . . . 50
8.4.2 12 Stores per Cluster . . . 50
8.4.3 Main 24-Stores-per-Cluster Run . . . 53
8.4.4 Cluster-Size Comparison . . . 56

9 Discussion **58**
9.1 Scope and Limitations . . . 58
9.2 Full-Instance Run Results . . . 58
9.3 Clustering . . . 59
9.4 Cost Factors . . . 59
9.5 Benefits of Store Charging . . . 59
9.6 Future Work . . . 59

10 An Operational Tool for Retailers **61**
10.1 Motivation . . . 61
10.2 Setup . . . 61
10.3 Input Data . . . 61
10.4 Running the Solver and Tracking Progress . . . 64
10.5 Reviewing results . . . 65
10.6 Applicability Beyond the Case Study . . . 65

11 Conclusion and Outlook **66**
11.1 Conclusion . . . 66
11.2 Answers to Research Questions . . . 66
11.3 Future Research . . . 67

List of Figures **68**

List of Tables **69**

Acronyms **70**

Bibliography **71**

1 Introduction

Replacing a diesel grocery-delivery fleet with battery-electric trucks is a planning problem in which long-term infrastructure decisions interact with uncertain daily demand. The chargers that support such a fleet are installed at fixed locations, typically at the warehouse and at the retailer's own stores, and the placement is expensive and hard to reverse once it is made. Daily demand, by contrast, varies considerably: a plan that works for an average day can fail on a difficult one. The thesis therefore studies the charger placement question as a worst-case planning problem, in which the placement is chosen so that the hardest observed demand day can still be served, and so that its operating cost is as low as possible.

For the thesis, real delivery data from a large German grocery retailer was available. The retailer at the time of recording the data was relying on diesel trucks. Before building the optimization model, it is necessary to understand what the data actually shows, in order to get the daily demand scenarios, retrieve the relevant vehicle size, and analyze the time needed for unloading the truck in order to have concrete values that can be relied on in the further process. The main planning question then becomes at which stores the retailer should install chargers.

Grocery delivery is a suitable case for electrification. The trucks regularly drive from a central warehouse to a set of retailer-owned stores which stay the same over a long period of time. The variance lies in the actual demand the stores request for each day. Also, not every store is served on every day. Since the trucks stop at the store for unloading, that idle time can be used without any further cost for charging the battery. Charging along the route extends the range of each truck and enables longer tours. Without it trucks would be forced to return to the warehouse or do a charging stop at an external charging provider, with elevated costs for waiting and the higher sold energy price. Another good reason for spreading out charges to a larger geographic area is that for a large fleet relying on charging all trucks at one central location in parallel could exceed the available grid connection power at that location, while getting enough power for only one charger at multiple locations is much more realistic.

The thesis models this problem as a two-stage robust optimization problem. In the first stage, the model needs to decide where to install the chargers and of which type they should be. In the second stage, the model optimizes for the best routing based on the actual demand scenario and the available chargers. The model tries to minimize the costs and remain feasible

for demand scenarios with the highest total cost.

After building the model and trying to run it on the real delivery data it was realized that the main challenge to get valid results is computational complexity. As the number of stores, necessary trucks and available scenarios increases, the model to optimize becomes very large, which does not give feasible solutions in a reasonable time. The solution provided in this thesis is therefore not a solver for the full retailer data, which guarantees optimal solutions, but rather a tool to make the Charging Station Placement Problem (CSPP) solvable for large instances. The algorithm is built and tested against the real-world data. Finally, a web application with a graphical user interface which incorporates the CSPP algorithm and provides methods to upload and sanitize instance data was created so that a retailer can run the model on their own dataset.

1.1 The Case for Electrifying Grocery Delivery

Grocery logistics in Germany is still almost entirely fossil-fuel-based. In 2024, food and beverages were responsible for 50 billion tonne-kilometres by domestic trucks [1], and in 2023, 97.8% of the transport sector still relied on fuels [2]. In 2023, CO_2 emissions from road freight were still 14.6% above their level in 1995 [3]. Electric trucks can not only lower emissions, but also reduce energy-cost volatility, since warehouse charging uses stable industrial electricity prices, while diesel prices are subject to volatile oil markets [4, 5, 6]. Several political measures have been put in place recently, which also favor the transition to electric trucks. KfW Zuschuss 441 subsidizes charging infrastructure with up to 900 EUR per charger and up to 45,000 EUR per location [7]. Electricity used in electric trucks can also be monetized via the THG-Quote [8], and zero-emission trucks remain exempt from the German truck toll until 30 June 2031 [9].

1.2 Research Questions

The thesis is organized around three research questions.

1. How can the historic retailer data be transformed into the two-stage robust CSPP which optimizes the charger placement for the worst observed demand days?
2. How can realistic values for the needed parameters be derived from the vehicle specifications and empirical data?
3. Which clustering method and runtime decisions make the CSPP solvable?

1.3 Contributions

The thesis makes six main contributions.

- It turns historical delivery records into optimizable instances with store location and distance matrices, unloading time estimates and daily demand scenarios.
- It formulates a two-stage robust CSPP with charger installation decisions, routing and charging decisions and load-dependent energy use.
- It finds realistic values for parameters used by the CSPP from technical data or historic records, or makes explicit assumptions about them.
- It implements various clustering methods, a geographic method that produces clusters which are close together, a store-count angular-slices method that cuts the map into equal slices around the warehouse, a demand-aware angular-slices method which balances equal store-demand instead of store-count, and a tour-containment method which keeps historical tours intact.
- It implements a three-stage pipeline, which finds robust solutions for the first stage, then evaluates each scenario against that solution and additionally implements a first stage reoptimization step which optimizes the global worst scenario across all clusters.
- It publishes a web-based tool that allows any retailer to run the model on their own dataset and get a robust solution.

1.4 Thesis Structure

Chapter 2 analyzes the historic retailer data and makes assumptions about unloading time, energy and infrastructure costs. Chapter 3 looks at related literature on electric vehicle routing, robust optimization and clustering methods. Chapter 4 formulates the robust CSPP formulation and methods used in preprocessing. Chapter 5 explains where the concrete parameter values come from. Chapter 6 describes the implemented algorithm, explained alongside a concrete example. Chapter 7 explains how bucket-based time limits were introduced based on empirical runtime calibration tests. Chapter 8 first compares the four different clustering methods on a reduced 96-store instance, then presents a varying cost parameter study and finally reports the results of the full 259-store instance runs with two different cluster sizes. Chapter 9 discusses the findings and limitations. Chapter 10 introduces the web application. Chapter 11 closes the thesis answering the research questions and suggesting future work.

2 Problem Setting and Data Basis

2.1 The Retailer Data

This chapter looks at the historic retailer data and the assumptions that can be made from the data. Some of the assumptions are straightforward. Others are only approximations.

The starting point is one month of real delivery records from a large German grocery retailer in March 2025. At the time when the data was recorded, the retailer still used diesel trucks. The raw records contain 28,924 delivery data entries. After filtering the data for inconsistencies, merging duplicate records, the dataset used in the thesis contains 6,775 tour stops, 259 stores and 2,186 tours across 31 demand scenarios.

The first thing which was looked at was how much load was carried in individual tours. The trucks start at the warehouse with the cargo for multiple tour stops, and return with an empty load. The median initial load per tour is about 9.5 tons. The average identified tour makes about 4.2 stops. This is why medium-sized distribution trucks with a load capacity of 15-20 tons were used in the thesis rather than heavy long-haul trucks with around 40 tons load capacity.

The second important part is the time spent at stores for unloading their cargo. If the truck is electric, this waiting time can be used for charging at no extra time cost and the truck can fill up its battery without adding a separate charging stop at an external charging station. This extends the daily range of the truck and allows for more tours and tours with more stops. This is the main operational reason why store chargers are considered in the first place.

The third observation are the daily demand scenarios. Not every store is served on every day and the demand also varies a lot for each store. This is one of the reasons, the robust planning approach in Chapter 4 is used. The charger plan should not be optimized for the average day, but rather keep the costs for the worst scenarios as low as possible.

2.2 Choosing Representative Electric Trucks

The thesis implements the CSPP for two electric trucks:

- the Mercedes-Benz eActros 300 with payload capacity $L = 17{,}700$ kg and battery capacity $C = 336$ kWh,

- the Volvo FM Electric with payload capacity $L = 16{,}200$ kg and battery capacity $C = 360$ kWh.

These two vehicles are already used by some grocery retailers who switched to an electric fleet and cover the observed load capacity in the observed tours. Their battery capacities are also large enough to keep the trucks for multiple stops, but still too small to eliminate the need for route-side charging. Both vehicles are also marketed for urban distribution and grocery delivery [10, 11]. The two vehicles not only have different payload and battery configurations but also the types of charger supported for each of them is different.

Other comparable electric trucks also exist, such as the Volvo FE Electric and FL Electric [12], the Renault Trucks E-Tech D and D Wide [13], and the MAN eTGL [14] but not considered in the thesis as they are currently less commonly used in German retail logistics. However the solution developed in the thesis can further be applied to these vehicles as well.

2.3 Unloading Time

To model charging at stores, the available free unloading time has to be estimated first. Unloading duration is estimated from 6,198 tour stops as a linear function of delivered weight:

$$\text{Duration (min)} = 11.51 + 0.004184 \cdot \text{Weight (kg)}.$$

Stops with missing or zero values were removed from the tour stop history. To remove outliers with very long waiting times, the upper 1% of durations was also filtered out.

Figure 2.1 shows the fitted regression.

The observed data is quite noisy, which could be explained by the fact that some records probably include small delays, and the delivery weight is certainly not the only factor determining stop duration. But still the direction shows a positive relationship between delivery weight and unloading time.

The energy that can be charged during a stop is then modeled as power provided by the charger, multiplied by the available time, with a linear efficiency factor [15]. This is only a planning approximation, the model does not reproduce the full charging curve, but since the trucks are required to stay above 20% State of Charge (SoC) and the trucks start with a battery charged to 80%, the linear approximation is sufficient for the purpose of this thesis. It simply turns the estimated stop duration into a usable amount of charged energy.

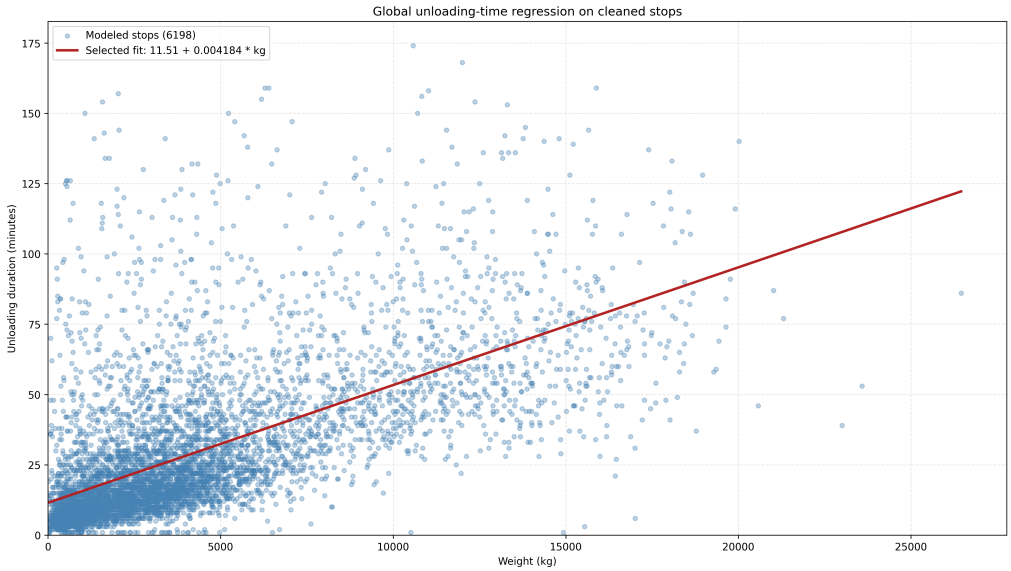

Figure 2.1: Unloading-time regression on 6,198 cleaned delivery stops

2.4 Energy Consumption and State-of-Charge Assumptions

Energy consumption is modeled as dependent on the current cargo load. A grocery truck starts the tour heavy loaded and becomes lighter after each delivery. The energy consumption per kilometre on an arc is modeled as a linear function of the current cargo load t:

$$P(t) = P_{\min} + \frac{t}{L}(P_{\max} - P_{\min}),$$

where $P_{\min}$ is consumption for an empty truck, $P_{\max}$ for a fully loaded truck, and L is payload capacity. The load is the main factor influencing the state of charge [16]. Other factors such as the terrain, the weather conditions, and the driver's driving style play a secondary role and are difficult to model and are therefore not considered. Refrigeration which also affects the load and energy consumption are also not modeled in the thesis. The numerical values for $P_{\min}$, $P_{\max}$, the SoC window, and related parameters are specified in Chapter 5. The full battery propagation including store charging variables is given in Chapter 4.

2.5 Charging Infrastructure Assumptions

2.5.1 Charger Types and Installation Costs

In the implementation for this thesis the warehouse is assumed to provide basic 22 kW AC charging in every configuration, because for overnight charging, slow charging is usually enough, and it removes pressure from the electrical grid which would otherwise exceed the

available grid connection and may require a costly upgrade to its infrastructure [17, 18].

The main decision in this thesis is which stores should receive chargers and which charger type should be used. Those are the first-stage decisions of the optimization model.

Store chargers extend the operating range of a truck, by using unloading stops that are already part of the tour. This can make tours feasible that would otherwise require a return to the warehouse or an external charging stop. Charging at selected stores no longer depends only on one central charging location.

2.6 Optimization Instances

With these assumptions fixed, the historical records are transformed into optimization instances. Each historical day represents one demand scenario. The instance data contains the active stores of that day, the demand weights, the distances between all locations, the charger types and the vehicle parameters. The 31 demand scenarios form the uncertainty set of the robust model, for which a charger placement decision is taken in the first stage. Afterwards, the plan optimizes routing across all observed scenarios. Chapter 4 formalizes this optimization problem.

3 Related Work

This chapter discusses the relevant literature for the charging station placement problem. It focuses on the parts that are actually used in the thesis: electric routing with charging decisions, two-stage robust planning, and clustering methods as a way to make large routing problems solvable.

3.1 Electric Vehicle Routing and Store Charging

Hiermann et al. [19] study an electric routing problem where the model also has to decide which vehicle types are used. This is relevant here because the route cannot be planned independently from the battery size, the available charging stations and the truck that is assigned to it. Schiffer et al. [20] state that electrification of commercial vehicle fleets in retail mid-haul logistics only makes sense when truck choice, battery range, routing, charging infrastructure, and fleet composition are evaluated together. Changing one of these parameters impacts the cost or even the feasibility of the others. While they focus mostly on fleet transformation and composition, this thesis uses the insights of it for store charging station placement under uncertain demand.

The main focus of this thesis is not primarily how to route the tracks on a given day, but where to install charging stations in advance.

Borlaug et al. [17] take into consideration possible limitations on grid connection. Charging many heavy-duty trucks in parallel at one location can lead to peak power usage in the local electricity grid which may require grid reinforcement. Therefore, store chargers are valuable for this thesis for two reasons. They use free unloading time for charging along the route and they also reduce the dependence on one central warehouse charging location.

3.2 Robust Optimization

The charging station placement problem has two stages. The charger decision is made before the actual demand day is known. Daily routing and charging decisions are made at a later point in time, once the demand scenario is observed. The robust objective then protects the plan against the hardest observed scenarios, instead of only optimizing for the average day.

The direct methodological starting point of this thesis is recent work from the TUM Chair of Optimization and Sustainable Decision Making, where this thesis was written. Goerigk et al. [21] propose the approximate scenario bracketing procedure (ASBP), a recent column-and-constraint generation approach for finite-scenario two-stage robust mixed-integer problems. The method is built for problems where second-stage solves are expensive. It is also built for cases where only a few scenarios are expected to really determine the robust solution. The main idea is to work with scenario upper and lower bounds. Weak scenarios can then be removed early, while exact solve time is spent on scenarios that still influence the worst-case cost.

This matches the CSPP studied in this thesis. For every demand scenario, the second stage contains a mixed-integer routing and charging problem. These are computationally very expensive solves. Solving all scenarios to a reasonable gap at every iteration would be too expensive for the retailer data. This thesis uses ASBP for the robust solution logic, since it keeps the solve focused on the scenarios that can still influence the worst-case charging station placement.

Rahmaniani et al. [22] give a broader overview of decomposition methods for two-stage optimization problems. Their review confirms that such models are often too large for a direct solve and that performance depends on bounds, warm starts and other ideas to accelerate computation. This supports the general direction of the thesis, which uses clustering to split the problem into smaller instances and uses available warm starts to speed up the runtime.

3.3 Clustering

Fisher and Jaikumar [23] use decomposition strategies to reduce routing complexity by splitting the problem before optimizing the routes. A large routing problem becomes much easier if it is divided into smaller subproblems. For this thesis, however, the split is not arbitrary. Kerscher and Minner [24] mention that clustering not only reduce problem size but should also preserve useful historical route structure. This use case is addressed by the tour-containment clustering method implemented in this thesis.

Clustering is a practical way to solve the robust CSPP at the scale of the available retailer data, since the full problem with all stores and scenarios is too large to solve in a reasonable time. The thesis compares several clustering methods, which are described in detail in Chapter 6.

While clustering improves the runtime of the algorithm, the downside is that it restricts which routes can be taken, which leads to a worse overall objective value.

3.4 Positioning of This Thesis

This thesis studies a retailer-specific charging station placement problem for electric grocery delivery under uncertain daily demand. The main decision is which stores should receive chargers before the demand day is known. The model uses the fact that trucks already stop at stores for unloading, so the forced waiting time can be used for charging. It also takes into consideration the practical limitation that charging many trucks at the warehouse can be constrained by the local grid.

The thesis implements a two-stage robust model using the ASBP together with various clustering methods. ASBP is used to handle the finite set of demand scenarios. Clustering is used to make the routing and charging problems small enough to solve. The contribution is therefore not only an electric routing model. It is about how charger placement, uncertain grocery demand, clustering and bounded-time robust solving fit together in one practical grocery delivery setting.

4 Charging Station Placement Problem

4.1 Problem Setting and Worst-Case Objective

The retailer chooses once where to install store chargers, and then operates deliveries every day under this fixed charger network. The open question for the model is which cost the charger decision should minimize. An average over all observed days would let a plan look good even if a few days are very expensive or even infeasible. On such heavy days the fleet may not finish its tours, and exactly these days drive the operational risk of the transition. The charger network should therefore be chosen so that the hardest observed day is still manageable.

Formally, let a denote a charger placement, $f(a)$ the one-time infrastructure cost, and $g^s(a)$ the operational cost on demand scenario $s \in S$. The worst-case daily operating cost is

$$z(a) \;:=\; \max_{s \in S} g^s(a),$$

and the planner solves

$$\min_{a} \; f(a) + z(a).$$

So the charger network is chosen so that the most expensive day becomes as cheap as possible, while investment cost is still taken into account. An empty placement is a valid outcome; it means that warehouse-only charging is already sufficient under the chosen assumptions. In some early 30-kilometre-radius experiments that are not reported in this thesis, this was indeed the case: all demand could be served with warehouse-only charging, so no store chargers were installed.

This is a *robust* objective in the sense of [22, 25]: the decision is protected against the worst realization inside an uncertainty set, without requiring a probability distribution over scenarios. Here the uncertainty set is the finite set S of historical daily demand patterns of the retailer. Each scenario is a real day that actually occurred in the past, and the resulting outcome is: the charger plan is feasible, and its cost is bounded, for every day in the dataset.

This gives a natural two-stage structure: the charger placement a is fixed once, and routing and charging are then chosen per scenario in response to a. The scenario-level subproblems are themselves mixed-integer routing-and-charging problems with payload, battery, and

charging constraints. The constraints $z \geq g^s(a)$ for all $s \in S$ link them to the placement a and push z up to the cost of the worst day.

The rest of the chapter describes this structure formally. Section 4.2 gives the mathematical formulation. Section 4.3 explains the two preprocessing steps that are part of the model definition. Chapter 5 then explains where the concrete parameter values come from, and Chapter 6 describes the solution method.

4.2 Mathematical Formulation

4.2.1 Problem Structure and Notation Mapping

The CSPP is modeled as a two-stage robust mixed-integer program with a finite uncertainty set. This class of problems, denoted as (2-RO) in [21], has the general form

$$\min_{x \in \mathcal{X}} \left(f(x) + \max_{s \in S} \min_{y^s \in \mathcal{Y}^s(x)} g^s(x, y^s) \right) \tag{4.1}$$

where x represents the first-stage decisions, y^s the second-stage decisions for realized scenario $s \in S$, and $\mathcal{X}, \mathcal{Y}^s(x)$ the feasible sets. For the CSPP, this mapping is:

- **First-stage decision** x: The charger installation vector a, where a_j^τ determines if a charger of type τ is deployed at store j.
- **First-stage objective** $f(x)$: The infrastructure investment cost $f(a)$.
- **Second-stage decision** y^s: The operational vector for scenario s, including routing (r^s), payload flow (t^s), recharging (p^s, p^s_{wh}), and scheduling variables.
- **Second-stage objective** $g^s(x, y^s)$: The scenario-specific operational cost $g^s(a, y^s)$, which depends on the installed infrastructure a via recharging limits and waiting times.

The robust objective combines the investment cost with the worst operational cost over all scenarios. This is needed because the charger locations are fixed before the actual demand scenario is known, while the routes and charging decisions are chosen afterwards.

4.2.2 Sets and Parameters

Network structure

i_0	single warehouse (depot)
J	set of stores
V	set of nodes ($V = \{i_0\} \cup J$)
A	set of arcs (subset of $V \times V$ with known distances)
l_{v_1,v_2}	length of arc (v_1, v_2) for $(v_1, v_2) \in A$

Warehouse parameters

κ_{wh}	charging speed of one warehouse charger (22 kW)
α_{load}	fixed loading-time component at warehouse (in hours)
γ_{load}	additional loading time per kg loaded at warehouse (in hours/kg)

Fleet parameters

$K_{\max}$	upper bound on the number of trucks; derived from worst-case demand (see Section 4.3.2)
$M_{\max}$	maximum number of tours per truck; fixed by the experiment setting `max_tours_per_truck`
L	vehicle load capacity (tons)
C	vehicle battery capacity (kWh)
c_0	target SoC (in kWh); overnight charging restores this level
$P_{\min}$	kWh consumption per km for empty truck
$P_{\max}$	kWh consumption per km for fully loaded truck
F	fixed cost per used truck

Charging station parameters

T	set of charger types
e^τ	fixed cost for opening a charger of type τ at any store node
κ^τ	charging speed of charger type τ (in kW)
d	cost per kWh
h	cost per hour of extra waiting time (beyond unloading)
M	big-M constant ($M = C$, the battery capacity)

Demand parameters

S	finite set of scenarios
β_j^s	demand of store j in scenario s
δ_j^s	unloading time at store j in scenario s (in hours), proportional to β_j^s

4.2.3 First-Stage Decision Variables

Variable	Domain	Description
a_j^τ	$\{0, 1\}$	1 if charger type τ is installed at store j, 0 otherwise

Note: In the computational implementation used for the experiments, the warehouse charger is treated as a fixed 22 kW AC charging option. The strategic first-stage decision is therefore the deployment of store chargers.

4.2.4 Second-Stage Decision Variables

Second-stage decision variables for scenario $s \in S$:

Variable	Domain	Description
Truck/tour activation variables		
y_k^s	$\{0, 1\}$	1 if truck k is used, 0 otherwise

Variable	Domain	Description
$u^s_{k,m}$	$\{0,1\}$	1 if tour m of truck k is used, 0 otherwise
Flow variables (per truck $k \in [K_{\max}]$, tour $m \in [M_{\max}]$)		
$t^{s,k,m}_{v_1,v_2}$	$\mathbb{R}_{\geq 0}$	tons of cargo transported on arc (v_1, v_2) for $v_1, v_2 \in V$
$r^{s,k,m}_{v_1,v_2}$	$\{0,1\}$	1 if arc (v_1, v_2) for $v_1, v_2 \in V$ is used, 0 otherwise
SoC variables (per truck $k \in [K_{\max}]$, tour $m \in [M_{\max}]$)		
$c^{s,k,m,\text{arr}}_j$	$\mathbb{R}_{\geq 0}$	SoC upon arrival at store j (before charging, in kWh)
$p^{s,k,m}_j$	$\mathbb{R}_{\geq 0}$	energy charged at store j (in kWh)
$c^{s,k,m}_{\text{dep}}$	$\mathbb{R}_{\geq 0}$	SoC when departing warehouse on tour (k, m) (in kWh)
$c^{s,k,m}_{\text{ret}}$	$\mathbb{R}_{\geq 0}$	SoC when returning to warehouse after tour (k, m) (in kWh)
$p^{s,k,m}_{\text{wh}}$	$\mathbb{R}_{\geq 0}$	energy charged at warehouse before tour (k, m) (in kWh)
Overnight charging variable (per truck $k \in [K_{\max}]$)		
$p^{s,k}_{\text{overnight}}$	$\mathbb{R}_{\geq 0}$	energy charged overnight to restore c_0 (in kWh)
Waiting time variables (per truck $k \in [K_{\max}]$, tour $m \in [M_{\max}]$)		
$\omega^{s,k,m}_j$	$\mathbb{R}_{\geq 0}$	extra waiting time at store j beyond unloading (in hours)
$\omega^{s,k,m}_{\text{wh}}$	$\mathbb{R}_{\geq 0}$	extra waiting time at warehouse beyond loading (in hours)

The kWh consumption per km for cargo load $t_{v_1,v_2}^{s,k,m}$ is given as:

$$P(t_{v_1,v_2}^{s,k,m}) = P_{\min} + \frac{t_{v_1,v_2}^{s,k,m}}{L}(P_{\max} - P_{\min})$$

4.2.5 Objective Function

The value of the first-stage objective f is given by the cost of installing chargers at stores:

$$f(a) := \sum_{j \in J} \sum_{\tau \in T} e^{\tau} \cdot a_j^{\tau}.$$

The second-stage objective g^s for a scenario $s \in S$ is given as:

$$g^s := \sum_{k=1}^{K_{\max}} \sum_{m \in \{1,\dots,M_{\max}\}} \sum_{j \in J} d \cdot p_j^{s,k,m} \tag{4.2}$$

$$+ \sum_{k=1}^{K_{\max}} \sum_{m \in \{1,\dots,M_{\max}\}} \sum_{j \in J} h \cdot \omega_j^{s,k,m} \tag{4.3}$$

$$+ \sum_{k=1}^{K_{\max}} \sum_{m \in \{1,\dots,M_{\max}\}} d \cdot p_{\text{wh}}^{s,k,m} \tag{4.4}$$

$$+ \sum_{k=1}^{K_{\max}} \sum_{m \in \{1,\dots,M_{\max}\}} h \cdot \omega_{\text{wh}}^{s,k,m} \tag{4.5}$$

$$+ \sum_{k=1}^{K_{\max}} d \cdot p_{\text{overnight}}^{s,k} \tag{4.6}$$

$$+ \sum_{k=1}^{K_{\max}} F \cdot y_k^s. \tag{4.7}$$

The six terms of the second-stage objective represent different operating costs. Term (4.2) is the electricity cost of charging at stores. Term (4.3) penalizes waiting time at stores beyond unloading time. Term (4.4) is the electricity cost of warehouse charging between tours. Term (4.5) penalizes extra waiting time at the warehouse. Term (4.6) forces overnight charging back to the required state of charge c_0. Term (4.7) is the fixed cost per used truck.

Installing store chargers increases the first-stage cost $f(a)$, but it can lower the second-stage cost g^s because trucks can then charge on their routes. This is the main trade off in the model. Store chargers have a practical advantage in the retailer setting: trucks already stop at every store to unload, so the unloading time can be used for charging at no extra time cost. Trucks recover energy along the route and have a longer effective daily range. This enables tours that would otherwise need an extra return to the warehouse just for charging or make a stop at an external charging station provider with elevated costs for waiting and electricity. Relying only on warehouse charging avoids the infrastructure investment, but tends to force shorter tours, more trucks, and more waiting time. For a large fleet there is a further problem: charging all trucks at the warehouse at the same time can exceed the capacity of the warehouse grid

connection, as discussed in Section 5.3. Distributing charging across store sites reduces the peak electrical load at the warehouse and relaxes this grid bottleneck. The robust objective therefore chooses the charger configuration that gives the lowest total cost across considered demand scenarios.

4.2.6 Constraints

In the following, $[n]$ denotes $\{1, \ldots, n\}$ for any $n \in \mathbb{N}$.

The main job of the second stage of the CSPP is to decide how many trucks and tours per truck are needed, to keep track of the battery state of charge, in which order the stores are visited, and where and how long the trucks charge. The charger locations are fixed from the first stage, and the routing and charging decisions have to work for every scenario. The formulation is first summarized in blocks before the full constraints are listed. Table 4.3 lists what each block enforces.

Table 4.3: Second-stage constraint blocks and what they enforce.

Block	What it enforces
Symmetry breaking	Trucks are used in index order, so the solver does not check equivalent solutions.
Truck and tour activation	A tour can only be used if its truck is active; tours of one truck are used in order without gaps.
Tour structure	Each tour needs to start and finish at the warehouse.
Flow constraints	Each store with positive demand is visited once. The trucks cargo load can never exceed its load capacity.
Energy constraints	The SoC drops along every driven arc. How fast it discharges depends on the cargo load $P(t)$ the truck carries on that arc.
Overnight charging	After the last tour of the day, every used truck is charged back to the target SoC c_0.
Charging constraints	A truck can only charge at a store if a charger is installed there. Between tours, the truck charges automatically at the warehouse.
SoC bounds	The battery has to stay between 20% and 100% of capacity.

Given a scenario $s \in S$ and a first-stage solution a, the feasible set $\mathcal{Y}^s(a)$ is defined by the following constraints:

The symmetry-breaking constraint orders trucks by index. Truck $k+1$ can only be active if truck k is already active. Without this rule, the solver would explore many equivalent truck permutations.

Symmetry breaking constraint:

$$y^s_{k+1} \leq y^s_k \qquad \forall\, k \in [K_{\max} - 1] \tag{4.8}$$

The truck and tour activation constraints link tours to trucks. Constraint (4.9) says that a tour can only be used if its truck is active. Constraint (4.10) says that tours of one truck are used in order, so tour $m+1$ is only used if tour m is also used. Together they remove gaps in the tour indices.

Truck and tour activation constraints:

$$u^s_{k,m} \leq y^s_k \qquad \forall\, k \in [K_{\max}], m \in [M_{\max}] \tag{4.9}$$

$$u^s_{k,m+1} \leq u^s_{k,m} \qquad \forall\, k \in [K_{\max}], m \in [M_{\max} - 1] \tag{4.10}$$

The tour structure constraints require each active tour to start and end at the warehouse. Constraint (4.11) forces exactly one arc from the warehouse to some store at the start of an active tour, and constraint (4.12) forces exactly one arc from some store back to the warehouse at the end. For inactive tours, both sums are zero. This reflects the depot-return structure of grocery delivery.

Tour structure constraints:

$$\sum_{j \in J} r^{s,k,m}_{i_0,j} = u^s_{k,m} \qquad \forall\, k \in [K_{\max}], m \in [M_{\max}] \tag{4.11}$$

$$\sum_{j \in J} r^{s,k,m}_{j,i_0} = u^s_{k,m} \qquad \forall\, k \in [K_{\max}], m \in [M_{\max}] \tag{4.12}$$

The flow constraints cover both the visits and the cargo on board. Constraint (4.13) forces each store with positive demand to be entered exactly once across all trucks and tours. Constraint (4.14) forbids visits to stores with zero demand. Constraint (4.15) is the routing flow balance: if a truck enters a store on a tour, it must also leave it on the same tour. Constraint (4.16) is the payload flow balance: the cargo arriving at a store equals the cargo leaving it plus the demand delivered there. Constraint (4.17) limits the cargo on any arc to the truck capacity L and also forces cargo to zero on arcs that are not used. The cargo-flow variable t therefore links routing with energy consumption via $P(t)$.

Flow constraints:

$$\sum_{k=1}^{K_{\max}} \sum_{m=1}^{M_{\max}} \sum_{v \in V \setminus \{j\}} r_{v,j}^{s,k,m} = 1 \qquad \forall j \in J : \beta_j^s > 0 \tag{4.13}$$

$$\sum_{k=1}^{K_{\max}} \sum_{m=1}^{M_{\max}} \sum_{v \in V \setminus \{j\}} r_{v,j}^{s,k,m} = 0 \qquad \forall j \in J : \beta_j^s = 0 \tag{4.14}$$

$$\sum_{v \in V \setminus \{j\}} r_{v,j}^{s,k,m} = \sum_{v \in V \setminus \{j\}} r_{j,v}^{s,k,m} \qquad \forall j \in J, k \in [K_{\max}], m \in [M_{\max}] \tag{4.15}$$

$$\begin{aligned} \sum_{v \in V \setminus \{j\}} t_{v,j}^{s,k,m} &= \sum_{v \in V \setminus \{j\}} t_{j,v}^{s,k,m} \\ &+ \beta_j^s \cdot \sum_{v \in V \setminus \{j\}} r_{v,j}^{s,k,m} \qquad \forall j \in J, k \in [K_{\max}], m \in [M_{\max}] \end{aligned} \tag{4.16}$$

$$t_{v_1,v_2}^{s,k,m} \leq L \cdot r_{v_1,v_2}^{s,k,m} \qquad \forall v_1, v_2 \in V, k \in [K_{\max}], m \in [M_{\max}] \tag{4.17}$$

The energy constraints propagate the battery state of charge along each tour. Constraint (4.18) sets the departure SoC of the first tour to the target level c_0, and to zero if the truck is unused. The pairs (4.19)–(4.20), (4.21)–(4.22) and (4.23)–(4.24) each enforce an equality in big-M form only on the arcs that are actually used: (4.19)–(4.20) links the departure SoC at the warehouse with the arrival SoC at the first store, (4.21)–(4.22) links consecutive store visits and also adds the energy charged at the previous store, and (4.23)–(4.24) links the last store with the return SoC at the warehouse. The pair (4.25)–(4.26) connects one tour to the next: the departure SoC of tour $m+1$ equals the return SoC of tour m plus the energy charged at the warehouse in between. The key point is that energy consumption depends on the cargo load via $P(t)$, so a loaded truck consumes more energy per kilometre than an empty one.

Energy constraints:

$$c_{\text{dep}}^{s,k,1} = c_0 \cdot u_{k,1}^s \qquad \forall k \in [K_{\max}] \tag{4.18}$$

$$\begin{aligned} c_j^{s,k,m,\text{arr}} &\leq c_{\text{dep}}^{s,k,m} - l_{i_0,j} \cdot P(t_{i_0,j}^{s,k,m}) \\ &+ M(1 - r_{i_0,j}^{s,k,m}) \qquad \forall j \in J, k \in [K_{\max}], m \in [M_{\max}] \end{aligned} \tag{4.19}$$

$$\begin{aligned} c_j^{s,k,m,\text{arr}} &\geq c_{\text{dep}}^{s,k,m} - l_{i_0,j} \cdot P(t_{i_0,j}^{s,k,m}) \\ &- M(1 - r_{i_0,j}^{s,k,m}) \qquad \forall j \in J, k \in [K_{\max}], m \in [M_{\max}] \end{aligned} \tag{4.20}$$

$$c_{v_2}^{s,k,m,\text{arr}} \leq c_{v_1}^{s,k,m,\text{arr}} + p_{v_1}^{s,k,m} - l_{v_1,v_2} \cdot P(t_{v_1,v_2}^{s,k,m}) + M(1 - r_{v_1,v_2}^{s,k,m}) \quad \forall\, v_1, v_2 \in J, k \in [K_{\max}], m \in [M_{\max}] \tag{4.21}$$

$$c_{v_2}^{s,k,m,\text{arr}} \geq c_{v_1}^{s,k,m,\text{arr}} + p_{v_1}^{s,k,m} - l_{v_1,v_2} \cdot P(t_{v_1,v_2}^{s,k,m}) - M(1 - r_{v_1,v_2}^{s,k,m}) \quad \forall\, v_1, v_2 \in J, k \in [K_{\max}], m \in [M_{\max}] \tag{4.22}$$

$$c_{\text{ret}}^{s,k,m} \leq c_{j}^{s,k,m,\text{arr}} + p_{j}^{s,k,m} - l_{j,i_0} \cdot P(t_{j,i_0}^{s,k,m}) + M(1 - r_{j,i_0}^{s,k,m}) \quad \forall\, j \in J, k \in [K_{\max}], m \in [M_{\max}] \tag{4.23}$$

$$c_{\text{ret}}^{s,k,m} \geq c_{j}^{s,k,m,\text{arr}} + p_{j}^{s,k,m} - l_{j,i_0} \cdot P(t_{j,i_0}^{s,k,m}) - M(1 - r_{j,i_0}^{s,k,m}) \quad \forall\, j \in J, k \in [K_{\max}], m \in [M_{\max}] \tag{4.24}$$

$$c_{\text{dep}}^{s,k,m+1} \leq c_{\text{ret}}^{s,k,m} + p_{\text{wh}}^{s,k,m+1} + M(1 - u_{k,m+1}^{s}) \quad \forall\, k \in [K_{\max}], m \in [M_{\max} - 1] \tag{4.25}$$

$$c_{\text{dep}}^{s,k,m+1} \geq c_{\text{ret}}^{s,k,m} + p_{\text{wh}}^{s,k,m+1} - M(1 - u_{k,m+1}^{s}) \quad \forall\, k \in [K_{\max}], m \in [M_{\max} - 1] \tag{4.26}$$

The overnight charging constraints refill the battery at the end of the day. Constraint (4.27) activates for every truck after the tour m that is its last used tour (so $u_{k,m}^{s} - u_{k,m+1}^{s} = 1$), and forces the overnight energy to be at least the gap between c_0 and the return SoC of that tour. Constraint (4.28) handles the case where tour $M_{\max}$ is itself the last tour. Constraint (4.29) forces the overnight energy to zero for trucks that are not used. Together they ensure that every operating day starts from the same battery level c_0.

Overnight charging constraints:

$$p_{\text{overnight}}^{s,k} \geq c_0 \cdot (u_{k,m}^{s} - u_{k,m+1}^{s}) - c_{\text{ret}}^{s,k,m} \quad \forall\, k \in [K_{\max}], m \in [M_{\max} - 1] \tag{4.27}$$

$$p_{\text{overnight}}^{s,k} \geq c_0 \cdot u_{k,M_{\max}}^{s} - c_{\text{ret}}^{s,k,M_{\max}} \quad \forall\, k \in [K_{\max}] \tag{4.28}$$

$$p_{\text{overnight}}^{s,k} \leq C \cdot y_{k}^{s} \quad \forall\, k \in [K_{\max}] \tag{4.29}$$

The charging constraints connect the first-stage charger decision with the second-stage operations. Constraint (4.30) limits the energy charged at a store by charger power times available time (unloading plus extra waiting), and also forces it to zero if no charger is installed there. Constraints (4.31) and (4.32) set warehouse charging and warehouse waiting to zero before the first tour, because that tour already starts at the target SoC c_0. Constraint (4.33) limits the warehouse charging energy before tour $m \geq 2$ by charger power times the available window (fixed loading component, load-dependent loading component, and extra waiting). Constraint (4.34) forces warehouse waiting to zero for tours that are not used.

Charging constraints:

$$p_j^{s,k,m} \leq \sum_{\tau \in T} \kappa^\tau \cdot (\delta_j^s + \omega_j^{s,k,m}) \cdot a_j^\tau \qquad \forall j \in J, k \in [K_{\max}], m \in [M_{\max}] \tag{4.30}$$

$$p_{\text{wh}}^{s,k,1} = 0 \qquad \forall k \in [K_{\max}] \tag{4.31}$$

$$\omega_{\text{wh}}^{s,k,1} = 0 \qquad \forall k \in [K_{\max}] \tag{4.32}$$

$$p_{\text{wh}}^{s,k,m} \leq \kappa_{\text{wh}} \cdot \left(\alpha_{\text{load}} \cdot u_{k,m}^s + \gamma_{\text{load}} \cdot \sum_{j \in J} t_{i_0,j}^{s,k,m} + \omega_{\text{wh}}^{s,k,m} \right) \qquad \forall k \in [K_{\max}], m \in \{2, \ldots, M_{\max}\} \tag{4.33}$$

$$\omega_{\text{wh}}^{s,k,m} \leq M \cdot u_{k,m}^s \qquad \forall k \in [K_{\max}], m \in \{2, \ldots, M_{\max}\} \tag{4.34}$$

The state-of-charge bounds keep the battery level between 20% and 100% of capacity. Constraint (4.35) enforces the 20% floor on arrival at a store that is actually visited. Constraint (4.36) enforces the same floor for the return SoC of each used tour. Constraint (4.37) forces the SoC after store charging to stay below full capacity C. Constraint (4.38) bounds the departure SoC by C for used tours and forces it to zero otherwise. The 20% floor avoids deep discharge.

SoC bounds:

$$c_j^{s,k,m,\text{arr}} \geq 0.2 \cdot C \cdot \sum_{v \in V \setminus \{j\}} r_{v,j}^{s,k,m} \qquad \forall j \in J, k \in [K_{\max}], m \in [M_{\max}] \tag{4.35}$$

$$c_{\text{ret}}^{s,k,m} \geq 0.2 \cdot C \cdot u_{k,m}^s \qquad \forall k \in [K_{\max}], m \in [M_{\max}] \tag{4.36}$$

$$c_j^{s,k,m,\text{arr}} + p_j^{s,k,m} \leq C \qquad \forall j \in J, k \in [K_{\max}], m \in [M_{\max}] \tag{4.37}$$

$$c_{\text{dep}}^{s,k,m} \leq C \cdot u_{k,m}^s \qquad \forall k \in [K_{\max}], m \in [M_{\max}] \tag{4.38}$$

4.2.7 The Full CSPP

The full model, reformulated as a minimization problem, then reads:

$$
\begin{aligned}
\min \quad & f(a) + z \\
\text{s.t.} \quad & \sum_{\tau \in T} a_j^\tau \leq 1 && \forall j \in J \\
& z \geq g^s && \forall s \in S \\
& (4.8) - (4.38) && \forall s \in S \\
& a \in \{0,1\}^{J \times T},\ z \in \mathbb{R}.
\end{aligned}
\tag{CSPP}
$$

The variable z is the worst-case daily operating cost from Section 4.1. The constraints $z \geq g^s$ for all $s \in S$ force z to be at least the cost of every scenario. Since the objective minimizes $f(a) + z$, the model tries to bring the value of z, down to $\max_{s \in S} g^s(a)$. The constraint $\sum_\tau a_j^\tau \leq 1$ prevents the model from installing more than one charger type at the same store location.

4.3 Instance Preprocessing

The CSPP algorithm also implements two preprocessing steps before the model can be built.

4.3.1 Split-Delivery Preprocessing

In the available retailer data some stores have a demand β_j^s that is larger than the truck capacity L. In this case, one truck can not deliver the complete demand of the store in one tour. Therefore, the store is replaced by $\lceil \beta_j^s / L \rceil$ pseudo-stores at the same location. Each pseudo-store has demand at most L. The routing model then treats these pseudo-stores as separate stores. The physical location stays the same, but the demand is split into parts that can each fit on one truck tour.

4.3.2 Fleet Sizing and Multi-Tour Modeling

The number of allowed trucks $K_{\max}$ and tours per truck on a day, $M_{\max}$ also has to be fixed before solving the model. Both are set tightly, so that the model does includes as few unused routing variables as possible.

$K_{\max}$ is derived from the worst-case total demand across all scenarios, divided by the per-truck daily capacity $L \cdot M_{\max}$, and multiplied by a safety factor of 1.5. The result is capped at $|J|$. $M_{\max}$ is set to 3 by default. This covers the observed truck tours in the historic retailer data. It also keeps the number small enough for the model to remain computationally tractable.

5 Parameter Calibration

The model in Chapter 4 still needs concrete values for the parameters before it can run experiments. This chapter tries to set them as realistic as possible.

The parameters come from three sources. Some values come from the real historical retailer data, for example store locations, distances, demand scenarios, and unloading-time estimates. Some values come from researching public vehicle and charging data. Other values are planning assumptions, for example the value of waiting time and the lifetime used for spreading out the investment cost over the years. The parameter study in Section 8.2 then tests the varying economic assumptions.

5.1 Vehicle and Energy Parameters

The experiments mostly use the Mercedes-Benz eActros 300 as the main vehicle type, with the Volvo FM Electric setup as a secondary option. These two models are among the most widely used electric trucks in the German grocery retail sector. The vehicle parameters determine how much energy the battery stores and how much load the truck can carry. The final reported runs use the Mercedes setup, where the model uses a battery capacity of 336 kWh and a cargo load limit of 17 700 kg. The implementation also contains a Volvo setup with a 360 kWh battery and a cargo load of 16 200 kg.

The battery policy defines which range of the battery capacity is treated as usable. Each truck starts with an initial state of charge of 80 %. The state of charge is never allowed to fall below 20 %. This 20 % to 80 % window is used because battery discharging behaves mostly linearly in this range. Furthermore, charging up to 80 % can also be modeled as a linear process, whereas charging above this threshold is heavily throttled by the vehicle's battery management system. Limiting the maximum charge to 80 % and preventing deep discharges below 20 % also significantly reduces battery degradation and promotes long-term battery longevity [26]. This follows the common electric-vehicle planning idea that not every nominal kWh should be treated as usable energy for driving [27, 15].

Table 5.1: Main physical parameters used in the computational study.

Parameter	Value	Role in the model
Battery capacity	336 kWh	available truck energy
Payload limit	17 700 kg	maximum delivered load per tour
Initial SoC	80 %	start level for each truck
Minimum SoC	20 %	safety reserve at stores and depot return
Vehicle type	Mercedes eActros 300	typical electric grocery truck

5.2 Cost Parameters

The objective contains charger investment cost and daily operating cost. The operating cost includes electricity, waiting time, warehouse charging, overnight charging, and a fixed truck-use cost. All costs are expressed in EUR.

Table 5.2: Baseline economic parameters.

Parameter	Baseline value	Interpretation
Electricity cost d	0.25 EUR/kWh	Commercial charging price used in the main runs.
Waiting cost h	50 EUR/h	Planning value for truck and driver idle time during charging. It is intentionally higher than a pure wage estimate.
Truck fixed cost F	254 EUR/day	Yearly truck cost divided by the number of operating days, plus the cost of one driver day.
Charger lifetime	10 years	Lifetime used to divide the charger cost evenly across the years.
Operating days	290 days/year	Approximate six-day operation.

The waiting cost is not meant to be a measured wage. It is a planning value for the time in which a truck and driver are blocked at a charger. This matters because a truck that waits at a charger cannot serve another tour, and long charging stops make the daily schedule less robust. For this reason, the waiting cost is one of the parameters varied in the sensitivity study.

Fixed truck cost. Each truck used on a given day adds a fixed cost to the objective value. This cost represents the share of the vehicle price per operating day, and the cost of one driver day. The used daily fixed cost per truck is

$$F_v = \frac{p_v}{n^{\text{truck}} D_{\text{year}}} + c_{\text{day}}^{\text{driver}}.$$

The first term distributes the vehicle price p_v over the truck's lifetime n^{truck} in years and the number of operating days per year D_{year}. The second term $c_{\text{day}}^{\text{driver}}$ is the cost of one driver day. The truck lifetime in our runs is set to 9 years, following the German tax depreciation standard [28]. The driver cost is based on typical German salary studies, and the vehicle price is taken from current European market listings. Together, these assumptions give a Mercedes fixed cost of about EUR 254 per operating day.

5.3 Charging Infrastructure

The charger decision in the model is made at the stores. For each store, the model can either install no charger or choose one charger type from a fixed set. The set contains six charger types, ranging from slow AC charging to faster and more expensive DC charging. This is needed because the experiments should show whether the model prefers cheap low-power chargers, high-power chargers, or a mix of both.

Table 5.3: Store charger types used in the final experiments.

Type	Rated power	Installation cost
AC 22	22 kW	7500 EUR
DC 40	40 kW	22 500 EUR
DC 50	50 kW	33 000 EUR
DC 90	90 kW	60 000 EUR
DC 120	120 kW	75 000 EUR
DC 150	150 kW	100 000 EUR

The warehouse charger is fixed at 22 kW AC. The main reason is for this is that keeps the needed electricity lower when all trucks of the fleet charge in parallel overnight. More importantly for this thesis, fixing the warehouse charger keeps the focus on the main infrastructure decision of at which stores chargers should be installed.

A 22 kW AC charger corresponds to a standard three-phase low-voltage connection [29], which is widely available.

Table 5.4: Fixed depot parameters.

Parameter	Value
Warehouse charger	fixed 22 kW AC
Overnight charging window	10 h
Warehouse loading time	11.51 min + 0.004184 min/kg
Warehouse loading-time factor	0.6

The cost sensitivity study examines how varying electricity cost, waiting cost, and charger

investment cost affects the final charger placements and the objective value.

6 Solution Algorithm

The transformed retailer instance is too large to solve as one single CSPP. That is why the solution method splits the stores into several clusters. Each cluster is then solved with the scenario addition method: the model starts with an empty set of demand scenarios and adds one scenario in each iteration. It is important to keep in mind that the result is not, in our implementation, a mathematical proof of global optimality for the complete retailer instance. Instead the goal of our algorithm is to produce useful charger placements within a predictable runtime.

This chapter explains the complete algorithm alongside one concrete example from the main full-instance run on the real retailer dataset. It first describes how the algorithm chooses charger locations and how it calculates the resulting cost for each scenario. The clustering method in this run was set to use 24 stores as target size and 28 stores as maximum size per cluster. The illustrative example is cluster 6, the hardest cluster in that run, which contains 28 stores and required about 21.96 wall-clock hours to solve Stage 1, which determined the charging station placement. Section 6.6 follows this cluster through the full pipeline.

6.1 Model Complexity

The main complexity of the model lies in the second stage of the model. For each scenario, the model contains routing, charging, waiting, battery, and truck-use variables. Since these variables depend on the number of stores, the necessary trucks, and routes, the total number of variables for n stores, K trucks, and M tours is roughly

$$N_{\text{var}}^{(2)} \approx 2n(n+1)KM.$$

The number of variables increases rapidly as more stores are added. For the full instance with 259 stores, the resulting model is therefore too large to solve just by increasing the time limit.

Even for the relatively small chosen cluster sizes, large Mixed-Integer Programming (MIP) gaps are observed. But the solver still finds useful solutions early, while the gap stays large. This is analyzed in the runtime calibration experiment in Chapter 7. The experiment shows that the objective value reaches a plateau early, while later solve time gives only marginal improvements to the incumbent.

6.2 Clustering

Clustering reduces the number of stores the solver must handle at the same time. Instead of solving one large robust CSPP for all stores, the algorithm solves a smaller model for each cluster. This is an approximation, because trucks are not allowed to drive from one cluster to another. It is still a necessary step, since the full problem cannot be solved in a reasonable time.

The thesis uses four clustering methods:

- *geographic* clustering, which groups stores that are close to each other on the map, using k-means as the main step;
- *angular slices (store-count)*, which cuts the area around the depot into pie slices with the same number of stores in each slice;
- *angular slices (demand-aware)*, which cuts the same kind of slices, but balances the total demand instead of the store count;
- *tour containment*, which tries to keep as many historical delivery tours intact as possible, so as few tours as possible are broken.

All four methods build the same kind of arc set in the end. The depot is connected to every store in both directions. An arc between two stores is only kept if both stores belong to the same cluster. With cluster assignment $C(j)$, the arc set is

$$A_{\text{cluster}} = \{(0, j), (j, 0) : j \in J\} \cup \{(i, j) : i, j \in J,\ i \neq j,\ C(i) = C(j)\}.$$

The full arc set is also stored for comparison, but the clustered CSPP stages use A_{cluster}. This is the step that makes the model smaller: store-to-store arcs across clusters are removed, while every store can still be reached from the depot.

6.2.1 Geographic clustering

The geographic method first runs k-means on the latitude and longitude of the stores. The depot is included when k-means is fitted, but its label is dropped afterwards. Only the store labels are kept. Including the depot makes the first cluster centers shift slightly toward it, which gives a better starting point than fitting on stores alone.

The first k-means labels are then improved by a small assignment MIP. Let J be the set of stores, K the number of clusters, and $d_{j,c}$ the straight-line distance from store j to k-means

center c. The reassignment model is

$$\min \quad \sum_{j \in J} \sum_{c=1}^{K} d_{j,c} x_{j,c} \tag{6.1}$$

$$\text{s.t.} \quad \sum_{c=1}^{K} x_{j,c} = 1 \qquad \forall j \in J, \tag{6.2}$$

$$n_{\min} \leq \sum_{j \in J} x_{j,c} \leq n_{\max} \qquad \forall c \in \{1, \ldots, K\}, \tag{6.3}$$

$$x_{j,c} \in \{0, 1\} \qquad \forall j \in J,\ c \in \{1, \ldots, K\}. \tag{6.4}$$

The number of clusters K is set from the target store count and the upper bound $n_{\max}$, so that every cluster can stay within the size limits. If the reassignment model has no feasible solution, the algorithm keeps the original k-means labels. The geographic method is therefore a distance-based method with hard size limits.

6.2.2 Angular slice clustering

The angular methods place the depot at the center and sort the stores by the angle at which they appear around the depot. This gives a circular order of stores from 0 to 2π.

Before the stores are split into clusters, the algorithm looks for the largest empty gap in this circular order, that is, the place on the circle where two neighboring stores are the furthest apart in angle. The cut between the first and the last cluster is placed inside this gap. The reason is simple: angles wrap around (2π is the same as 0), so a fixed cut at angle 0 could fall in the middle of a tight group of stores and split it for no good reason. Cutting in the largest gap avoids this and makes sure that the boundary between two clusters lies in an area with few stores.

In the store-count variant, the sorted stores are then split into K connected groups with almost the same number of stores in each group. Each cluster covers one slice around the depot, and neighboring clusters cover neighboring slices.

The demand-aware variant uses the same circular order, but it does not split by count alone. Instead, it uses dynamic programming. Let the sorted stores be $j_1, \ldots, j_n$, and let q_j be the largest demand of store j across all scenarios. The dynamic program looks for connected groups that respect the size limits and keep the total demand of each cluster as close as possible to the target demand $\bar{q} = \sum_{j \in J} q_j / K$:

$$\min \sum_{c=1}^{K} \left(\sum_{j \in G_c} q_j - \bar{q} \right)^2,$$

where $G_1, \ldots, G_K$ are connected parts of the sorted order and $n_{\min} \leq |G_c| \leq n_{\max}$. If no feasible split exists under the size limits, the algorithm falls back to the equal-count split. This method keeps the same pie-slice shape, but tries to avoid clusters that have a much higher worst-case demand than others.

6.2.3 Tour-containment clustering

Tour-containment clustering uses historical delivery tours instead of coordinates or demand. The raw data is grouped by tour name and matched to the internal store indices. Repeated stops within the same tour are removed, and tours with fewer than two stores are dropped. Tours that visit the same set of stores are merged into one pattern. The number of original tours that share the pattern becomes its weight, so a tour pattern that appears often counts more than a pattern that appears only once.

Let J be the set of stores, K the number of clusters, and $\mathcal{T}$ the set of merged tour patterns that are small enough to fit into one cluster:

$$\mathcal{T} = \{t \subseteq J : |t| \geq 2,\ |t| \leq n_{\max}\}.$$

Each tour $t \in \mathcal{T}$ has weight w_t, equal to how often the pattern appears after merging. The model is

$$\begin{aligned}
\max \quad & \sum_{t \in \mathcal{T}} \sum_{c=1}^{K} w_t v_{t,c} && && \text{(TOUR-CONTAINMENT)} \\
\text{s.t.} \quad & \sum_{c=1}^{K} x_{j,c} = 1 && \forall j \in J, && (6.5) \\
& n_{\min} \leq \sum_{j \in J} x_{j,c} \leq n_{\max} && \forall c \in \{1, \ldots, K\}, && (6.6) \\
& v_{t,c} \leq \frac{1}{|t|} \sum_{j \in t} x_{j,c} && \forall t \in \mathcal{T},\ c \in \{1, \ldots, K\}, && (6.7) \\
& x_{j,c} \in \{0, 1\} && \forall j \in J,\ c \in \{1, \ldots, K\}, && (6.8) \\
& v_{t,c} \in \{0, 1\} && \forall t \in \mathcal{T},\ c \in \{1, \ldots, K\}. && (6.9)
\end{aligned}$$

The variable $x_{j,c} = 1$ means that store j is placed in cluster c, and $v_{t,c} = 1$ means that the full tour pattern t is inside cluster c. The linking constraint has the same effect as a separate constraint $v_{t,c} \leq x_{j,c}$ for each store j in t: since $v_{t,c}$ is binary, it can only be one when every store of the tour is in the same cluster. Using one combined constraint per tour and cluster keeps the model smaller than writing one constraint per store, tour, and cluster. Tour patterns larger than $n_{\max}$ are removed before solving, because they cannot fit into any cluster.

Unlike the angular and geographic methods, this model does not aim directly at compact regions. The objective rewards keeping recurring tours complete inside a single cluster. Compactness only enters through the original tour shapes and the cluster size limits. After the assignment is found, the same depot-plus-within-cluster arc set is built.

The comparison in Section 8.1 shows that demand-aware angular slices give the best results, so the final runs in Chapter 8 use this method. In the example below, the stores are split into 11 clusters of about 24 stores each, and cluster 6 is followed through the algorithm.

6.3 Stage 1: Scenario Addition with Bound-Based Pruning

6.3.1 Algorithmic idea

Each cluster has 31 historical demand scenarios available. Adding all of them at once as separate constraint blocks in one large robust MIP would make the model too complex to solve. Instead, Stage 1 follows the approximate scenario bracketing procedure for two-stage robust mixed-integer problems [21], developed in recent work by the TUM Chair of Optimization and Sustainable Decision Making. This method only considers a small group of active scenarios, called $D \subseteq S$, at a time. At each outer step, the algorithm solves the main model (the restricted master problem) using only the scenarios in D. The remaining scenarios in $S \setminus D$ are then checked using short heuristic MIP instances. Only scenarios that can still change the worst case are added to D in the next step. In this way, the algorithm focuses on the most critical scenarios and avoids spending time on those that are clearly not the worst case.

The main loop for a cluster works as follows:

1. Solve the main model using only the scenarios in D to find a charger plan.
2. Run heuristic and bound updates on every scenario in $S \setminus D$ to maintain a lower bound $\mathrm{LB}[k]$ and an upper bound $\mathrm{UB}[k]$.
3. Remove scenarios that are clearly not the worst case. This happens if their maximum cost is already lower than what the main model expects or if they are less expensive than another scenario.
4. Find the most difficult remaining scenario and add it to the active set D.
5. Stop the loop when all scenarios have been handled, the target error margin is reached, or the time limit is up.

Let the master solution have first-stage cost $\tilde{f}$, second-stage bound $\tilde{z}$, reported relative MIP gap γ, and target gap p with $\gamma \leq p$. The adjusted z-bound from [21] is

$$z_{\text{bound}} = \frac{1-\gamma}{1-p}\,\tilde{z} + \frac{p-\gamma}{1-p}\,\tilde{f}. \tag{6.10}$$

This value is the cost level above which a scenario still matters for the master under the target gap p. These two rules use this bound to drop scenarios:

- **z-bound rule.** Drop scenario k if its highest possible cost $\text{UB}[k]$ is below the z-bound. This scenario cannot be more expensive than the current worst case that the solver is already okay with, so adding it would not change the plan.
- **Dominance rule.** Drop scenario k if its highest cost is below the lowest possible cost of another scenario. Another scenario is guaranteed to be more expensive, so scenario k can never be the worst case.

If every unselected scenario is either added to D or removed by these rules, Stage 1 stops.

6.3.2 Example: Cluster 6

The first outer step starts with no scenario in D, and runs heuristics on all 31 scenarios to select the initial worst scenario. After this step, every unselected scenario has an initial upper bound of about EUR 2032 (from the master second-stage relaxation), while only a few have a tight lower bound because most second-stage probes have not finished yet. The most critical scenario is added to D, and the master is solved again.

Over the next steps, D grows and the upper bounds for individual scenarios are improved by repeated heuristic and second-stage probes. Whenever a candidate's upper bound drops below the master's used z-bound, it is permanently removed. For Cluster 6, this happens to four scenarios. Their final upper bounds are between EUR 302 and EUR 306, while the master's used bound at the end is $z_{\text{bound}} \approx$ EUR 328. None of these four scenarios is added to D, because doing so cannot improve the robust decision under the target gap.

The cluster 6 solve stops after 28 first-stage iterations. At this point, 27 scenarios are active in D_6 meaning that they are considered for the charging station placement optimization. Only four scenarios, $\{9, 16, 23, 30\}$, are removed by the z-bound rule. The reached MIP gap is 55.4 %. The first-stage plan installs 15 store chargers. This corresponds to a first-stage charger investment cost share of $\tilde{f} \approx$ EUR 228.79 per day. The reported master's worst-case second-stage objective is bounded above by about EUR 2632.

This example shows that the charger plan is found early, but the solver then spends much of the remaining time validating whether any other scenario can still make this plan worse.

This supports the decision in Chapter 7 to limit the time for each step instead of running one very long calculation. A high reported relative MIP gap does not automatically mean that the charger plan is poor.

6.4 Stage 2: Scenario Evaluation

Stage 2 starts from the charger plan found in Stage 1. The charger locations are fixed, and the algorithm then calculates the routes and charging for every demand scenario. This step also includes scenarios that were not actively optimized for when the chargers were placed in Stage 1. This step is necessary to evaluate the plan across the full retailer instance, not only inside one cluster.

For each cluster C and scenario s, the algorithm computes the operating cost g_C^s. The total cost of a scenario is then the sum of these cluster costs:

$$g_{\text{sum}}^s = \sum_C g_C^s.$$

The scenario with the highest value of g_{sum}^s is the global worst case. Stage 2 calculates the costs that are needed in Stage 3 to get the concrete values of how much each cluster contributes to each scenario's total cost.

6.5 Stage 3: Reoptimization

Stage 3 takes the global costs of Stage 2 and identifies the global worst-case scenario. A scenario that has not been considered by Stage 1 in a cluster can still become important when the costs of all clusters are added together. Stage 3 identifies these global worst-case scenarios and re-solves clusters that did not include that scenario in Stage 1, preserving the already included set of scenarios and adding the new one.

6.5.1 Loop structure

Stage 3 uses one global active set A. This set contains the difficult scenarios that have been found at the start of Stage 3 and after each iteration. It starts empty, so $A = \emptyset$. Each step then checks the current plan, adds one relevant scenario if needed, and re-solves only the clusters that missed this scenario in Stage 1. The loop has the following structure.

Step a (`eval_before`). The current plan is tested against all scenarios. The scenario with the highest total cost is selected as $s^\star$. In the first iteration, this does not require a new evaluation

because the same costs were already calculated in Stage 2.

Step b (convergence check). The algorithm then checks whether another reoptimization step is still useful. The loop stops if the current worst scenario has been included in an earlier step. It also stops if every cluster already included this scenario in its Stage 1 scenario set.

Step c (expand A). If the loop continues, the scenario $s^\star$ is added to the global active set A. This marks the scenario as globally relevant for Stage 3.

Step d (`cluster_solve`). Only the clusters that did not include $s^\star$ in their Stage 1 scenario set are reevaluated. The other clusters keep their existing plans. This produces a new candidate plan without repeating work for clusters that already considered the global worst case.

Step e (`eval_after`). The candidate plan is evaluated again. To keep this step faster, the algorithm only re-evaluates the clusters whose plans actually changed. The costs of the unchanged clusters can be reused.

Step f (accept or reject). The candidate plan is accepted only if it does not increase the total cost. If it improves the result, it becomes the current plan. If it makes the result worse, the previous plan is kept. In that case, the loop only continues if the worst scenario has changed, because a different scenario may still point to a useful correction.

Termination. The loop stops when no further improvement is found or after a fixed maximum number of steps. The best plan found during the loop is used as the final output. This acceptance rule is important because Stage 3 is meant to correct missed global scenarios, not to make an already good plan worse.

6.5.2 Reoptimization Behavior

The full retailer instance run with a 24-store cluster size stops after the first iteration. The most expensive scenario was scenario 27. This scenario was already included in the Stage 1 active sets of 10 out of the 11 clusters. Stage 3 therefore had to re-solve only one cluster. The plan of that cluster did not change the charging station placement, which made the loop terminate. This means that Stage 1 had already found the most important scenarios for the larger clusters and including the scenario in the remaining cluster's active set did not change the charging station placement.

The run with the 12-store cluster size in Chapter 8 behaves differently. With smaller clusters, relevant global scenarios are missed more often. In that run, Stage 3 needed several iterations

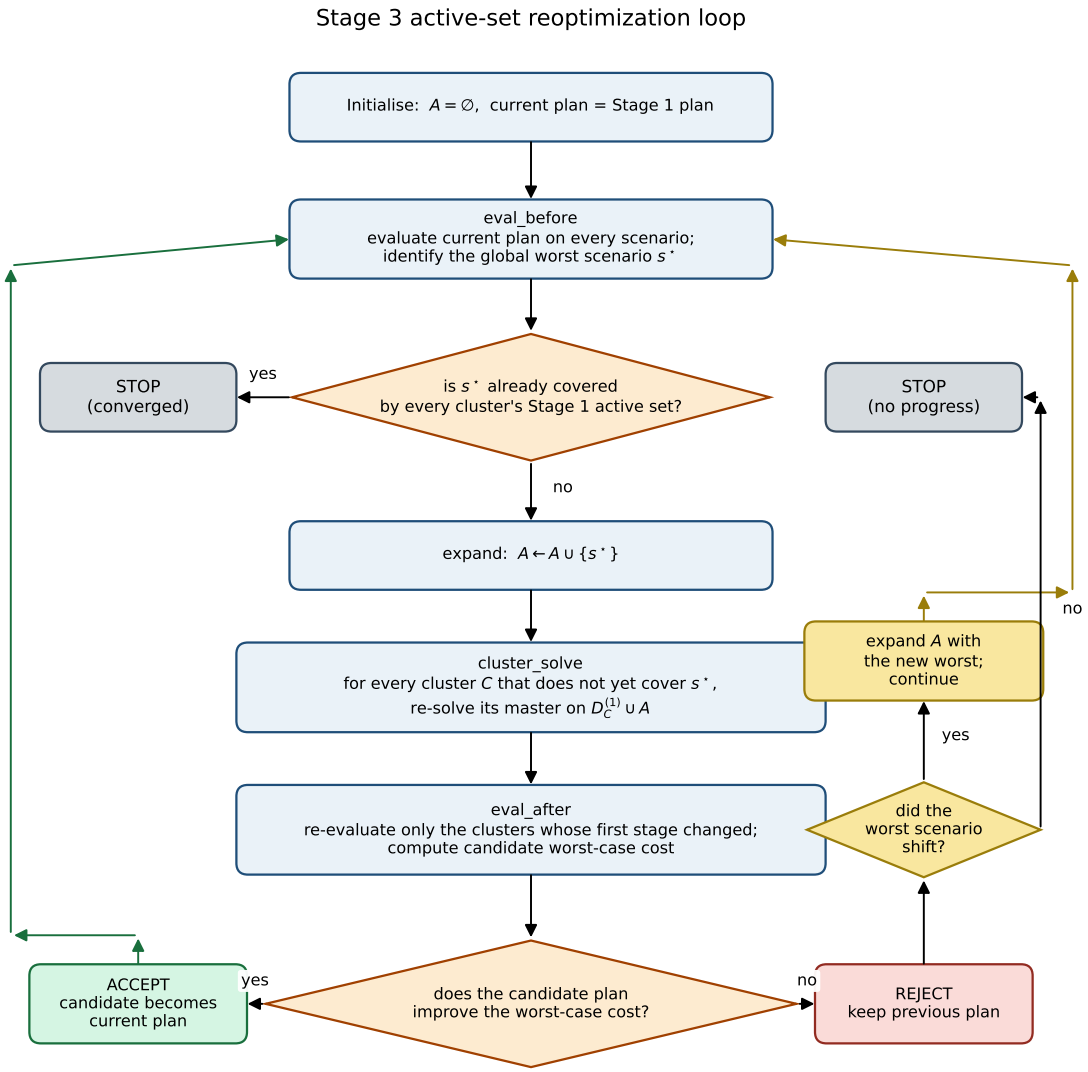

Figure 6.1: Stage 3 active-set reoptimization loop. The candidate plan is accepted only if it does not increase the total cost. If the plan is rejected but the worst scenario has changed, the loop continues with a new scenario in A.

to improve the result. The acceptance rule is important in this setting because it allows Stage 3 to make useful corrections while keeping the best plan found so far.

6.6 Pipeline Summary

The full pipeline is split into four main steps:

1. **Cluster the stores.** The stores are divided into smaller groups with the chosen clustering method.
2. **Stage 1.** Each cluster is solved separately. The stage finds a charger plan and saves the active set of scenarios that are relevant for this cluster's solution.
3. **Stage 2.** The operating costs are calculated for every scenario in every cluster. This identifies the global worst-case scenario after the cluster costs are added together.
4. **Stage 3.** Clusters that did not include the global worst-case scenario are re-solved. A new plan is accepted only if it improves the worst global objective.

The cluster 6 example shows how this works in practice. In Stage 1, the algorithm decides on the charging placement by including relevant scenarios and removing scenarios that cannot worsen it. In Stage 2, this fixed plan is evaluated against all scenarios across all clusters. Scenario 27 is then the most expensive scenario for the full instance. Since scenario 27 was already part of the Stage 1 active set for almost all clusters, Stage 3 only has little work left in this example. The final plan that includes cluster 6 has a total daily worst-case cost of about EUR 17 378.72 (Chapter 8).

7 Runtime Calibration

The solution algorithm from Chapter 6 needs several MIP time limits. These limits decide how long the solver continues to optimize on every Stage 1 iteration and on every Stage 2 evaluation. This is important because a complete run consists of many separate solves and can take many hours. If a limit is too high, the solver spends time on a
glsmip solve whose incumbent is barely improving. If a limit is too low, the solver may stop before it has found a good solution.

This chapter explains how the time limits used in the final CSPP runs were chosen. The calibration first tests how the target cluster size changes the runtime of the Stage 1 charger placement. It then checks how quickly Stage 2 evaluations reach a stable objective value. This results in a bucket policy that assigns time limits based on the expected difficulty of each cluster.

7.1 Stage 1 cluster size calibration

7.1.1 Setup

The first test looks at the size of the Stage 1 clusters. Stage 1 was run with target cluster sizes of 12, 18, and 24 stores. For this test, the algorithm did not use runtime limits and ran until completion without any time limit in place. The goal was to see how much harder the first-stage charger-placement problem gets as the cluster grows and when more worst scenarios are added.

7.1.2 Results

The runtime for the 13 test clusters grew clearly as the clusters got larger. For the 12-store target, Stage 1 terminated on average after 39.3 min per cluster. For the 18-store target, the average was 56.5 min. For the 24-store target, the average per cluster was 163.6 min.

This has direct consequences for the full retailer instance. Small clusters reach optimality in a few minutes, but the larger clusters used in the final runs do not. Without time limits, a complete run would not be solvable in a reasonable time.

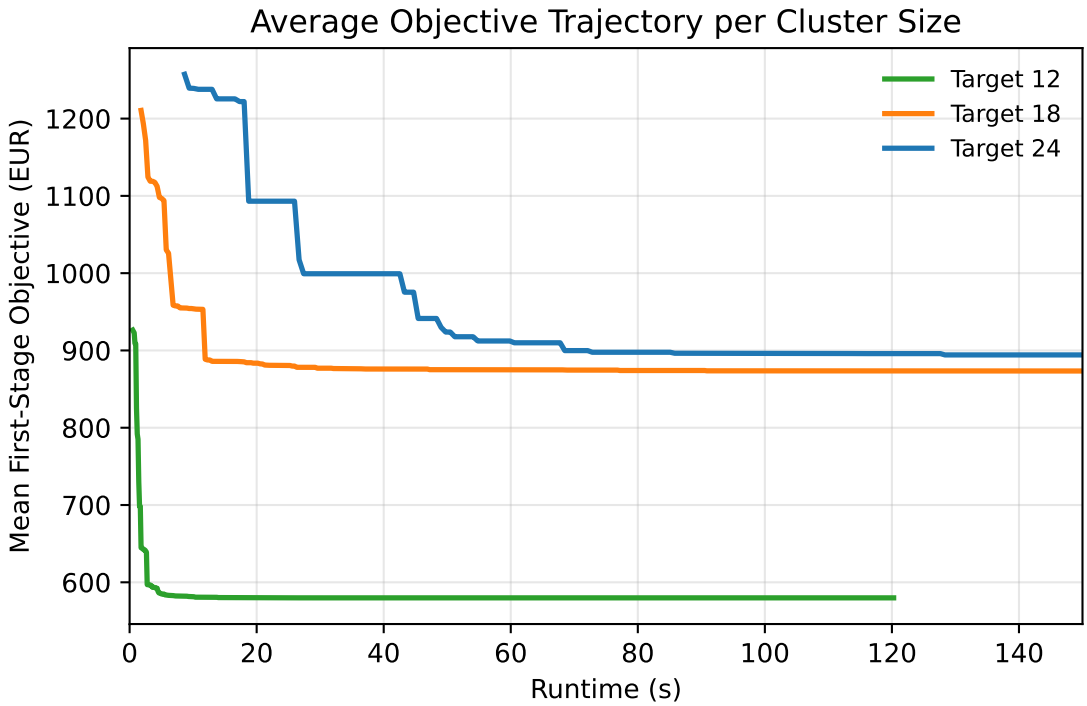

Figure 7.1: Stage 1 objective trajectory

7.2 Stage 2 Plateau Experiment

7.2.1 Setup

The second experiment looks at the Stage 2 scenario evaluations. Its results were retrieved from the same runs as the Stage 1 calibration. For each cluster-size group the objective value over time was tracked. These trajectories were then averaged over clusters of the same size.

7.2.2 Results

The Stage 2 results also show that larger clusters need more time before the evaluation value becomes stable. For the 12-store target, the median time for when the objective was within one percent of its final value was about 35 s. For the 18-store target, the median was about 49 s. For the 24-store target, the median increased to about 184 s.

This supports also using different Stage 2 time limits for different cluster sizes. The goal is again to avoid waisting time on easy evaluations, because they usually stabilize quickly. At the same time, the larger evaluations should still be able to continue solving when they are still improving.

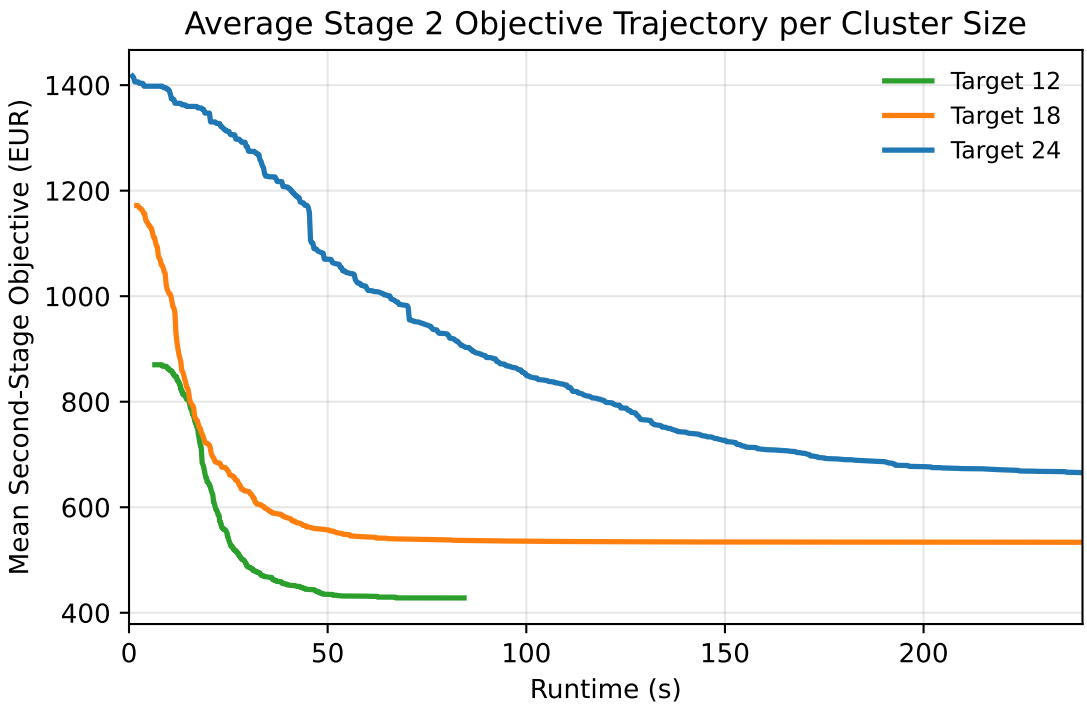

Figure 7.2: Stage 2 objective trajectory.

7.3 Total Number of MIP Solves

The time limits matter because the clustering creates a large number of individual MIP solves. Let C be the number of clusters, S the number of demand scenarios, and I_3 the number of Stage 3 reoptimization iterations that are evaluated. The planned workload is then counted as follows:

- Stage 1 solves one first-stage master problem for each cluster and scenarios that the algorithm adds to the active set D. The ASBP scenario addition removes scenarios whose upper bound is dominated, so the realized count per cluster is between zero and S. The continuation rule allows up to three additional Stage 1 rounds per cluster when the master still improves its objective value or did not found any feasible solution, which can increase the count further for up to $3C$. The Stage 1 entries in Table 7.1 report the master-iteration counts measured from the actual runs.
- Stage 2 fixes the charger plan and evaluates each cluster under each scenario. This gives $C \cdot S$ second-stage MIP solves.
- Stage 3 reoptimizes selected first-stage decisions and evaluates the new candidate. One iteration requires up to $C + 2CS$ MIP solves: one reoptimization master MIP solve per

cluster (C solves), and one second-stage evaluation per (cluster, scenario) pair both before and after the change ($2 \cdot C \cdot S$ solves). The total over all reoptimization iterations is therefore at most $I_3 \cdot (C + 2CS)$. The realized count is typically much smaller, because the eval-before of the first iteration reuses Stage 2 results and the eval-after only re-runs scenarios for clusters whose first-stage solution actually changed.

Table 7.1 applies these expressions to the two full-instance runs. The 12-store run already needs hundreds of Stage 2 MIP solves and thousands of potential Stage 3 MIP solves. The 24-store run did not need a successful reoptimization step. For this run, the Stage 3 number in the table only describes the first reoptimization attempt before the algorithm stops.

The time limits therefore have to be short enough to avoid spending time on stalled solves. They also have to be flexible enough for unexpectedly hard evaluations. The Stage 2 and Stage 3 counts in the table are planning counts for the MIP solve workload. The implementation can reduce the realized Stage 3 count through cached evaluations and by skipping clusters that are already covered by the active scenario set.

Table 7.1: Number of MIP solves for the two full-instance runs. Stage 1 reports the master iterations measured from the run logs (the 24-store run additionally used 18 continuation rounds across its clusters); Stage 2 is the planned $C \cdot S$ count; Stage 3 is the planned $I_3 \cdot (C + 2CS)$ count. The Stage 3 count for the 24-store run is only the first reoptimization attempt, after which the algorithm stops because no improvement is needed.

Run	C	S	I_3	Stage 1	Stage 2	Stage 3
12 stores per cluster	22	31	3	274	682	4158
24 stores per cluster	11	31	1	174	341	693

7.4 Parallel Execution

The large number of MIP solves also makes parallel execution important. Many parts of the algorithm are independent. In Stage 1, different clusters can be solved at the same time. In Stage 2, each cluster–scenario pair can be evaluated independently. In Stage 3, the cluster re-solves and the scenario evaluations can also run in parallel within one iteration.

The implementation uses a fixed processor budget based on the available processor cores. Each running MIP solve receives a fixed number of Gurobi threads. In the final implementation, each parallel MIP solve uses four Gurobi threads. Therefore, a budget of B processor threads allows at most $\lfloor B/4 \rfloor$ MIP solves to run at the same time. This prevents the algorithm from starting more multi-threaded Gurobi solves than available processor cores.

Stage 1 and the Stage 3 cluster re-solves use separate worker processes for different clusters. Stage 2 uses a thread pool for all cluster–scenario pairs. The cluster-sum evaluation in Stage 3 uses the same parallelization logic. Parallelization only changes the wall-clock runtime, not the objective value that is reached.

The runtimes in Chapter 8 refer to wall-clock runtime.

7.5 Bucket Policy

The calibration results show that small clusters on average need less time than larger clusters. The algorithm therefore introduces several runtime buckets. Before a cluster is solved, it is assigned to a bucket based on its size, its distance from the warehouse, and the average number of active stores. Each bucket has a different time limit for Stage 1 and Stage 2. If the solver is still improving the objective value, or didn't find any feasible solutions when the time limit is reached, it can receive extra time by using the continuation rule as described below.

Table 7.2: Shared runtime bucket definition.

Bucket	Definition
`xsmall`	stores ≤ 12
`small`	stores ≤ 14
`medium`	stores ≤ 16, avg_depot_km < 15, mean_active_stores < 8.5
`hard`	stores ≤ 20, avg_depot_km < 30, mean_active_stores < 10
`xhard`	all remaining cases

Table 7.3: Time limits used by the bucket policy.

Bucket	Stage 1 [s]	Stage 2 [s]
`xsmall`	120	45
`small`	180	60
`medium`	240	120
`hard`	300	180
`xhard`	360	240

The Stage 1 limit applies to each iteration. The Stage 2 limit applies to each scenario evaluation. If a solve reaches its time limit without an incumbent, or if the incumbent was still improving late in the round, the implementation allows up to three continuation rounds. A late improvement means that the last objective value improvement occurred after 80 % of the current round's time limit. Each continuation increases the cumulative time budget by one additional base limit. For example, an `xhard` Stage 1 master solve starts with 360 s but

can receive up to 1440 s in total if necessary. An `xhard` Stage 2 evaluation starts with 240 s and can receive up to 960 s. Stage 3 uses the same continuation rule for re-solves.

This continuation rule is a safeguard for unexpectedly hard evaluations and avoids too conservative base limits for every solve. It allows the algorithm to spend more time on cases where no feasible solution has been found or where the solver is still making progress.

7.6 Time Limits in Action

In the main run with 24 stores per cluster, all 11 clusters are classified as extra hard (`xhard`) because they are large. In the run with 12 stores per cluster, the clusters fall into different smaller buckets, as cluster store counts, distances and active store numbers vary. The results in Chapter 8 show that these time limits work well for the final experiments. Good charger plans are found early, and stay within the time limits. The time limits therefore behave as intended.

8 Results

This chapter presents all experimental results. Section 8.1 compares four clustering methods on a smaller 96-store instance to select the method used in the full experiments. Section 8.2 then explores how cost parameters influence charger placement on a single cluster from the same 96-store instance. Section 8.3 describes the setup for the full-scale experiments, and Section 8.4 reports the final results on the 259-store instance.

8.1 Clustering Method Comparison

Choosing the right clustering method is an important decision. The way stores are grouped into clusters determines which routes the trucks can take. Since trucks cannot travel between clusters, a poor choice at this stage cannot be repaired later. This section compares four clustering methods on a smaller instance. Based on the results, the best-performing method is selected for the full-scale experiments.

8.1.1 Setup

The clustering methods are tested on a smaller problem with 96 stores and 8 demand scenarios. The stores are split into clusters of about 12 stores each, resulting in about 8 clusters per method. The clustering methods which are compared are geographic clustering, tour containment, store-count angular slices, and demand-aware angular slices. All tests use the same settings and the same algorithm described in earlier chapters. The goal is to identify the method that achieves the lowest total cost for the worst-case scenario.

8.1.2 Results

Figure 8.1 reports the cluster-size distribution produced by each method, and Table 8.1 summarizes the cost outcomes after Stage 2 and reoptimization.

8.1.3 Interpretation

Demand-aware angular slices perform best. Before reoptimization, the tour-containment and geographic methods appeared to perform better. After Stage 3, however, the ranking

Figure 8.1: Cluster-size distribution in the 96-store clustering comparison. Store-count angular slices produce exactly balanced clusters, but this does not produce the best robust result.

Table 8.1: 96-store clustering comparison. The columns "Avg. cost", "Stage 2 worst", and "Worst after reopt." are daily operating costs in EUR; "Worst after reopt." is the robust objective after Stage 3.

Method	Cluster sizes	Avg. cost	Stage 2 worst	Worst after reopt.	Chargers
Angular slices (demand-aware)	14,13,16,16,9,10,10,8	3565.20	4620.44	4027.17	32
Geographic	9,15,16,16,11,15,1,13	3413.49	4619.03	4209.18	24
Tour containment	8,16,15,7,15,4,16,15	3247.24	4338.11	4338.11	29
Angular slices (store-count)	12,12,12,12,12,12,12,12	3745.26	4984.62	4668.28	21

changed: the demand-aware angular-slices method reached the lowest worst-case cost and is therefore the best choice. This final post-reoptimization result is the relevant comparison criterion, since the full algorithm uses all three stages.

Equal cluster sizes are not the most important factor. The store-count angular-slices method created eight clusters with exactly 12 stores each. Although this seems like a fair split, it produced the highest cost. Equal store counts are therefore less important than demand, distance, and route structure. This method can result in very unbalanced demand clusters. When many high-demand stores fall into the same cluster, this cluster becomes much harder and dominates the worst-case outcome.

Geographic clustering creates distant clusters. Geographic clustering reached a competitive final cost. In earlier test runs on the full retailer instance, geographic clustering also produced very dense clusters far away from the warehouse. These were infeasible because the allowed maximum number of trucks could not serve all stores.

Tour containment can be strong on average if the historic tours are already good. Tour containment had the lowest average cost, but did not improve significantly after reoptimization.

Reoptimization significantly improved the results for most methods. It improved the final worst case for demand-aware angular slices by EUR 593.27, for geographic clustering by EUR 409.85, and for store-count angular slices by EUR 316.34. This step can therefore completely change which method gives the best overall result.

8.1.4 Choice for the Final Experiments

The final experiments in Section 8.4 use the demand-aware angular-slices method. It produces the best final robust result, avoids creating extreme one-store clusters, and does not rely on the historical tour data required by tour containment. It also fits the depot-centered layout of the retailer, splitting stores by their angle around the warehouse while balancing their demand.

The clustering comparison itself uses 12 stores per cluster, while the final experiments report results for both 12 and 24 stores per cluster. Larger clusters take longer to solve but yield a more robust plan. Further comparisons of clustering methods at the full retailer scale are left for future research, as noted in Chapter 9.

8.2 Cost Sensitivity Study

Chapter 5 chooses realistic costs for electricity, waiting time, and chargers. These costs are not fixed in practice. Electricity prices depend on the current market conditions, driver wages can change over time, and charger prices change as the technology develops. The experiment in this section varies each of these costs and looks at how the charger placement decision is affected. The point is to see how the model behaves as cost assumptions change.

8.2.1 Setup

The sensitivity tests use a single cluster of 16 stores. Each test varies one cost parameter and keeps the others fixed. The varied parameters are electricity cost, additional waiting time cost, and the costs for buying and installing a charger. Only the first-stage charger placement is evaluated, since we are interested in how the charging station placement adapts to cost changes.

8.2.2 Results

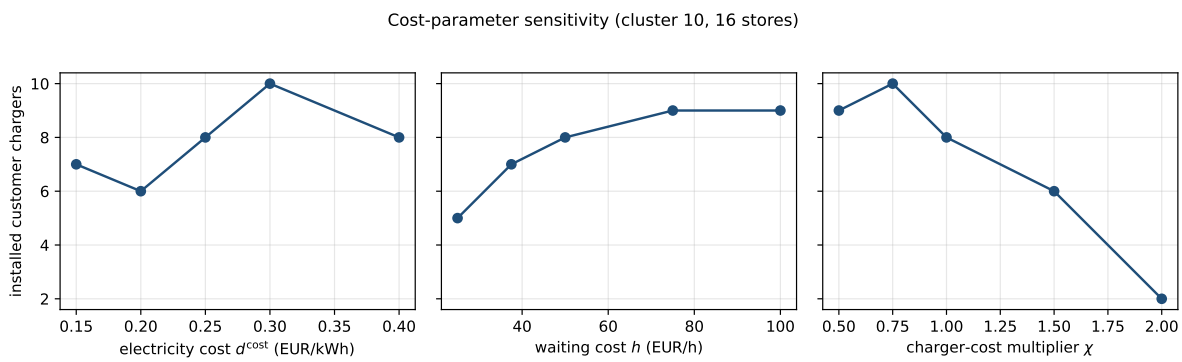

Figure 8.2: Number of installed store chargers as different costs are changed.

Waiting time cost. The number of installed chargers increases as the cost for waiting longer than the free unloading time increases. For $h = 25, 37.5, 50, 75$, and 100 EUR/h, the number of installed chargers is 5, 7, 8, 9, and 9. This is expected. More store charging capacity reduces the need for waiting beyond the unloading time.

Charging station cost. The cost for buying and installing chargers has the strongest effect. For $\chi = 0.5, 0.75, 1.0, 1.5$, and 2.0, the number of installed chargers is 9, 10, 8, 6, and 2. Overall, the result moves in the expected direction. When chargers are more expensive, the model usually tends to install fewer of them. The small deviations from a strictly monotone pattern,

especially the increase from $\chi = 0.5$ to $\chi = 0.75$ are more likely due to the time limits or the remaining MIP gaps, rather than a change in the overall trend.

Electricity cost. The electricity cost does not impact the number of installed chargers. For $d^{\text{cost}} = 0.15, 0.20, 0.25, 0.30$, and 0.40 EUR/kWh, the number of installed chargers is 7, 6, 8, 10, and 8. The reason for this might be that the energy required for the routes stays the same. Changing the electricity price affects the operating cost but not necessarily how many chargers are installed.

Key Takeaway. To get the best results from this model, accurate costs for driver waiting time and charging equipment are the most important inputs. Electricity prices affect the total cost but do not significantly change the charger plan in this range.

8.3 Full-Instance Runs

This section describes the setup for the two full retailer instance runs which include all stores from the dataset. The earlier tests led to the implemented time limits and provided the best-performing clustering method. The two runs use the exact same data and settings, while only the number of stores per cluster differs.

8.3.1 Data and Settings

The processed dataset contains 259 stores served from a single warehouse and 31 demand scenarios. They both use the Mercedes-Benz eActros 300 truck.

Table 8.2: Setup for the full-scale experiments.

Item	Setting
Stores	259
Demand scenarios	31 historical days
Vehicle	Mercedes-Benz eActros 300
Clustering method	Demand-aware angular slices
Main run	24 stores per cluster
Secondary run	12 stores per cluster

24 stores per cluster. The stores are split into 11 clusters. Larger clusters take more time to solve, but they preserve more of the natural delivery structure. This experiment provides the main results.

12 stores per cluster. The stores are split into 22 clusters. The secondary comparison run tests whether smaller clusters make the problem easier to solve and how the final objective value is affected.

Comparing the two exposes the trade-off between runtime and the quality of the final plan. The results are presented in Section 8.4.

8.3.2 Scope and Consistency

All settings in these runs, such as the clustering method and cost parameters, are the default baseline values fixed in earlier chapters. The only difference between the two experiments is the cluster size, which allows a fair comparison.

Earlier runs determined which parts of the algorithm were too slow and needed a speed-up mechanism and also which decomposition strategy performs best. They shaped the setup of the final experiments. The cluster-size comparison led to the introduction of the bucket-based time limits, and we chose the demand-aware angular slices because it was the best-performing clustering method from the comparison runs.

8.4 Full-Instance Results

This section presents the final results for the full 259-store experiments. Following the plan from Section 8.3, the 12-stores-per-cluster run is examined first, followed by the 24-stores-per-cluster run. Both experiments completed the full run successfully.

8.4.1 Store Layout and Demand

Figure 8.3 shows the layout of the 259-store instance and the highest daily demand at each store.

8.4.2 12 Stores per Cluster

Because the clusters are small and all clusters fall in the smallest time-limit bucket, Stage 1 finishes relatively fast, taking about 5.22 wall-clock hours. Shorter, predictable runtime is the primary benefit of using smaller clusters.

Charger placement

This run installs 94 chargers at a total investment cost of EUR 3 622 000. The plan tends to install slower, cheaper chargers compared to the main run, with many more 22 kW AC chargers and fewer medium-power DC chargers.

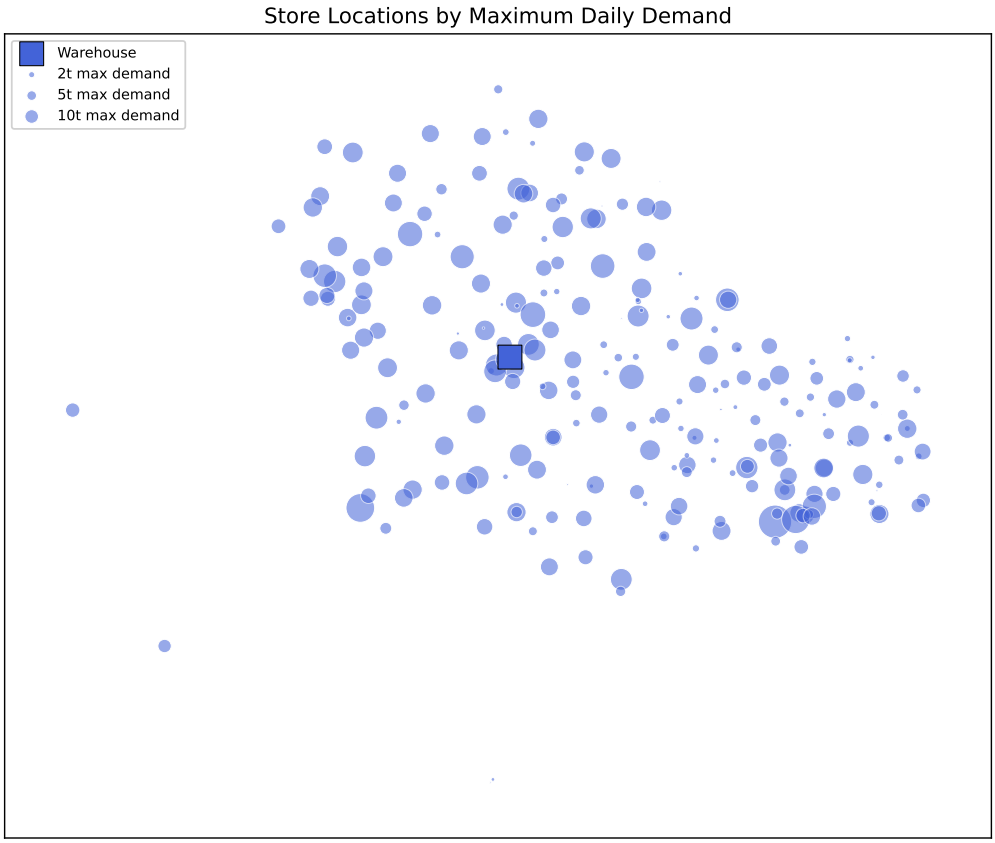

Figure 8.3: Full 259-store instance. Size shows highest daily demand.

Total cost before correction

After Stage 2 evaluates all 31 scenarios, the highest daily operating cost is EUR 23 278.78.

Reoptimization

The algorithm performs three iterations in this run: the first two improve the global worst-case cost, and the third one is rejected because it increased the worst-case cost.

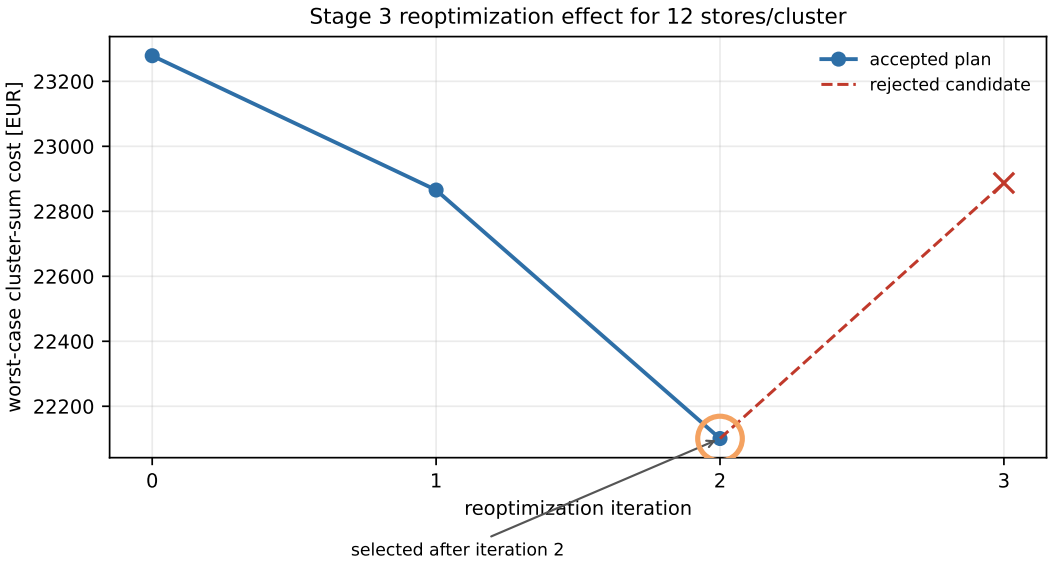

Figure 8.4: Stage 3 reoptimization trace for the 12 stores per cluster run. Iteration 3 increases the objective value and is rejected, Iteration 2 is selected.

Iteration 1. Before this step, the worst scenario is scenario 21 at a total cluster-sum cost of EUR 23 278.78. The reoptimization set A becomes $\{21\}$. Eleven clusters did not include the scenario in their Stage 1 active set and are solved again, adding the new scenario to it and reevaluating their charging station placement. Ten of those clusters change their charger configuration. The objective value after evaluating each second stage with the new chargers drops to EUR 22 865.47. The candidate is therefore accepted.

Iteration 2. The global worst scenario is now scenario 13, and the reoptimization set A is now $\{13, 21\}$. Fifteen clusters are solved again, and ten of them change their placement. The worst-case cost drops again to EUR 22 100.73. The candidate is accepted and becomes the new best plan.

Iteration 3. The global worst scenario shifts to scenario 20, and A grows to $\{13, 20, 21\}$. Nine clusters are solved again. The candidate plan is then evaluated against all scenarios, and its worst-case cost is EUR 22 887.18, which is higher than the iteration-2 cost. Because iteration 3 does not improve the objective, the candidate is rejected and the loop stops. The iteration-2 plan is the final result.

Reoptimization improves the isolated cluster plan by about 5 %. The acceptance rule discards iteration 3 as intended.

Limitations

Even though this experiment is fast in Stage 1, reoptimization is more likely needed, which also requires significant runtime. More importantly, the final cost with the smaller 12-store clusters is about 27 % higher than in the main 24-store result. Smaller clusters restrict the available delivery routes, so this is expected.

8.4.3 Main 24-Stores-per-Cluster Run

The 11 cluster sizes of this run range from 20 to 28 stores. Because they are larger, they are harder to solve, and Stage 1 becomes the biggest computational bottleneck.

Runtime

The clusters together require 150.46 CPU hours to solve. Because we ran the experiment on a 64-core machine and used up to 32 of them, multiple clusters could solve in parallel. Wall-clock runtime was 54.51 hours. The time-limit rules from Chapter 7 keep these long solves bounded.

Charging station placement

This run installs 95 chargers at a total investment cost of EUR 4 805 000. Unlike the previous run, charger types are more evenly distributed: 13 at 22 kW AC, 17 at 40 kW DC, 20 at 50 kW DC, 9 at 90 kW DC, 19 at 120 kW DC, and 17 at 150 kW DC. The higher number of high-power DC chargers is the main reason for the large investment cost.

Evaluation

The highest scenario cost is EUR 17 378.72 while the average cost is about EUR 11 463. This is a substantial improvement over the 12-store cluster result and confirms that larger clusters yield a better plan.

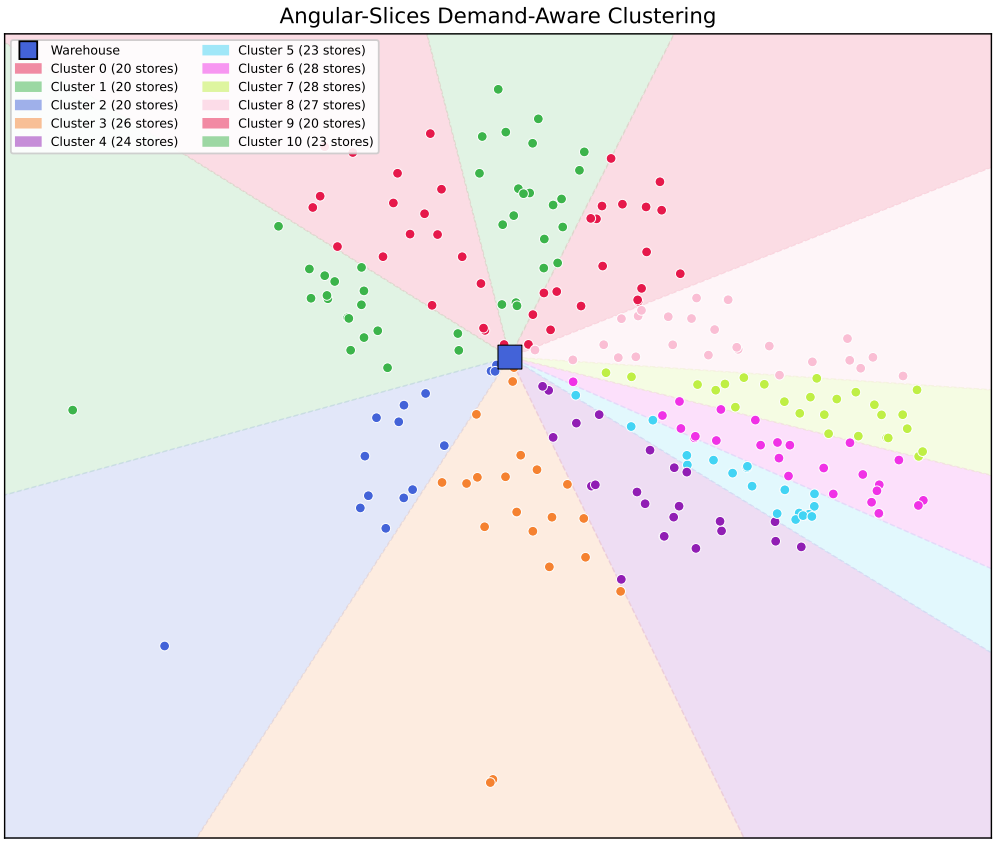

Figure 8.5: Clustering for the main experiment (11 clusters).

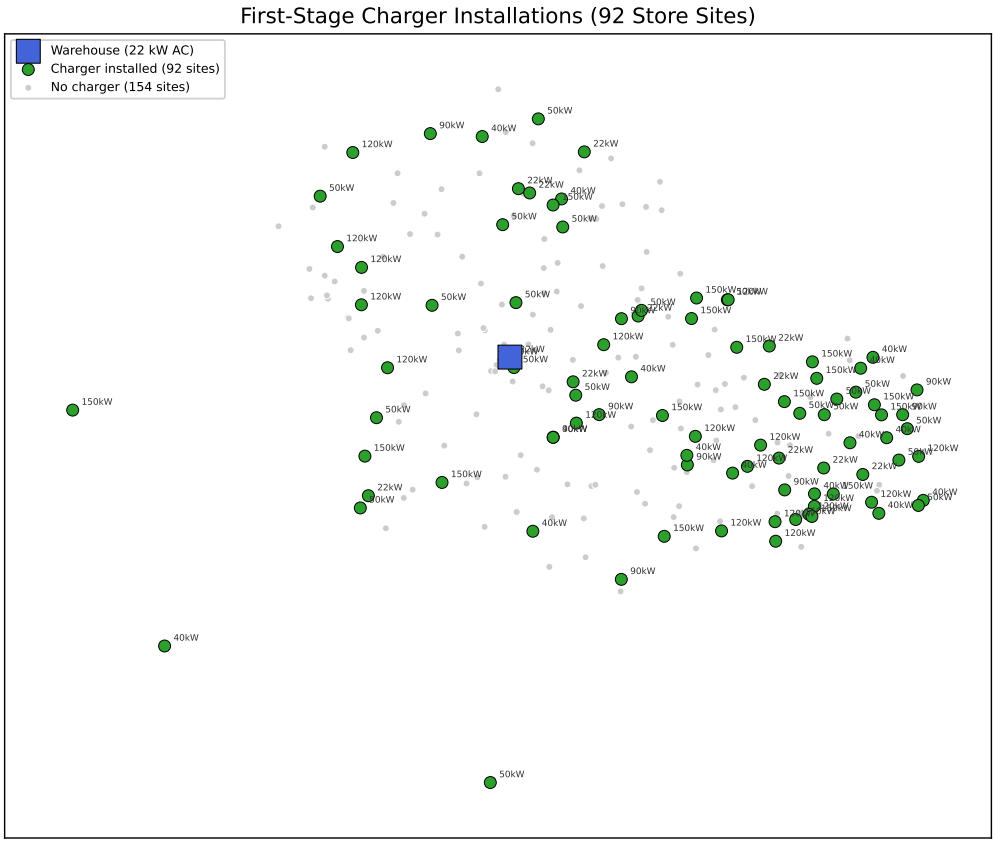

Figure 8.6: Installed store chargers in the main experiment.

Reoptimization

In this experiment, the global worst case after Stage 2 is scenario 27 with a total cost of EUR 17 378.72. Stage 3 adds this scenario to the reoptimization set. Ten of the eleven clusters had already included it in their Stage 1 active sets. Only cluster 9 is solved again. This solve does not change the first-stage charger placement, so the loop stops immediately.

8.4.4 Cluster-Size Comparison

Table 8.3 compares the results from both experiments. It is important to distinguish the one-time cost of buying and installing the chargers from the daily cost of making deliveries. The final results focus on minimizing this daily delivery cost.

Table 8.3: Comparison of the completed full-instance runs. Runtime is wall-clock time per stage. Costs are EUR per worst evaluated scenario.

Metric	12 stores per cluster	Main 24-stores-per-cluster run
Stores	259	259
Scenarios	31	31
Clusters	22	11
Installed store chargers	94	95
Store-charger investment cost	3,622,000	4,805,000
Stage 1 wall runtime (h)	5.22	54.51
Stage 2 wall runtime (h)	2.24	1.81
Stage 3 wall runtime (h)	27.77	0.11
Stage 2 worst-case cost	23,278.78	17,378.72
Final worst-case cost	22,100.73	17,378.72
Stage 3 improvement	5.06 %	0.00 %

Investment cost. Both runs install almost the same number of chargers (94 vs. 95). The 24-stores run is about EUR 1.2 million more expensive in investment cost, since it installs more high-power DC chargers. Figure 8.7 shows the distribution of charging stations. The 12-stores run leans heavily on cheap 22 kW AC chargers, while the 24-stores run installs more medium- and high-power DC chargers (56 of 95 chargers at 50 kW or above, vs. only 29 in the 12-stores run). This is consistent with findings in the cost-sensitivity study (Section 8.2): once routes get longer and waiting beyond unloading time becomes necessary, fast DC chargers reduce the driver's waiting cost.

Site-level overlap. Comparing the two plans store by store reveals that even though the number of installed chargers is roughly the same, only 42 of the chosen store locations are the

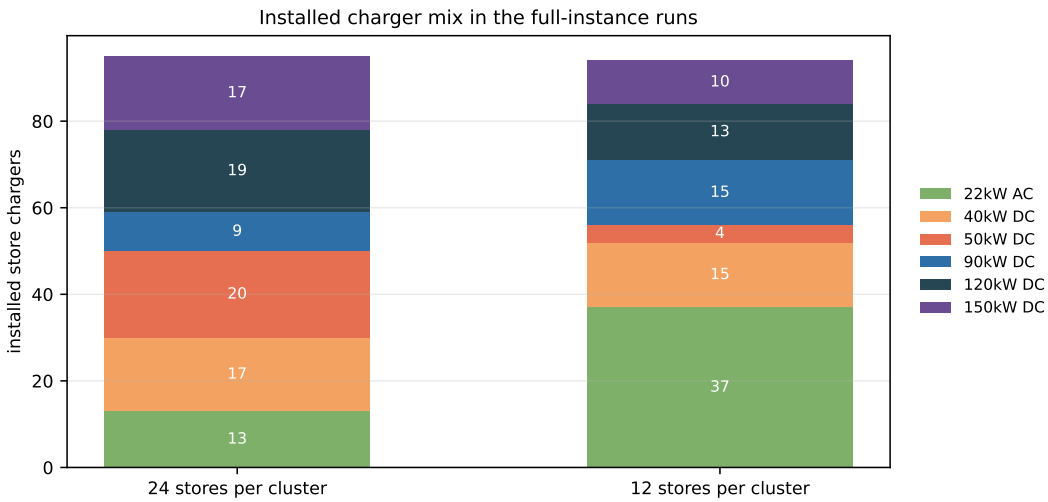

Figure 8.7: Installed charger types in both full-instance runs. Total counts are nearly identical (94 vs. 95), but the smaller cluster run favors low-power chargers.

same (about 45 %). Among the 42 shared locations, only 5 receive the same charger type in both runs. Cluster boundaries therefore determine not only *how many* chargers are placed but also *which stores* are chosen, which confirms that cluster size is a first-order design decision.

The results show a clear trade-off. Smaller clusters are faster to solve initially but lead to a higher final cost. Larger clusters require more computation but yield a better objective value. This is the expected outcome: larger clusters offer more routing flexibility and share charging capacity across more stores. The recommendation is therefore that the cluster size should be chosen based on the available hardware, picking the largest size that still solves within an available time budget. To support this choice in practice, the web application (Section 10.4) provides a runtime estimate based on the chosen number of stores per cluster and available processor cores. The 24-stores-per-cluster run, which achieves a daily worst-case cost of EUR 17 378.72, is the largest size tested on the available setup. The smaller cluster experiment is included to illustrate how cluster size affects the quality of the results.

9 Discussion

9.1 Scope and Limitations

The model is designed to support the decision of where to install chargers and ensure that the resulting charging station placement is robust against the most difficult scenarios.

- *Fixed cluster boundaries.* Routing beyond cluster boundaries in this setting is not possible and is not considered by the charging station placement.
- *Time-limites.* Because time limits are used, the results are not a guaranteed to be mathematically optimal.
- *Simplified warehouse charging.* A fixed charging setup is assumed at the warehouse; a mix of different charger types at the warehouse would probably make sense. This is, however, not explored in this thesis.
- *Fixed travel times.* Travel times are only calculated on distances and do not account for other factors such as traffic, weather, or road construction.
- *No specific time windows.* Store opening hours and exact appointment times are not considered.
- *No battery aging costs.* The cost of batterycapacity loss is not considered.

Despite these limitations, the results provide useful insight for charging infrastructure planning.

9.2 Full-Instance Run Results

The main experiment with 24 stores per cluster produces the strongest result. The final plan installs 95 chargers and reaches a worst-case daily operating cost of EUR 17 378.72. This is the primary finding of the study.

The experiment with 12-store clusters while being computationally faster, but the final daily cost is higher because routing options are restricted. Choosing the right cluster size has to consider the main trade-off between runtime and solution quality.

The smaller experiment shows that the Stage 3 reoptimization works as expected and becomes more relevant as the number of clusters increases. Its initial worst-case cost was EUR 23 278.78 and Stage 3 reduced it to EUR 22 100.73. In the main experiment, Stage 3 was not required because the global worst scenario was already covered by almost all clusters during Stage 1. Larger clusters therefore identify difficult scenarios earlier.

9.3 Clustering

Clustering is necessary for making the full instance solvable, but it also has drawbacks, such as restricting delivery routes. The 96-store comparison supports the use of demand-aware angular slices as the best choice, since balanced demand clusters have the best robust outcome. The geographic method came close in cost, but early test runs shows that it gets worse as the instance size increases, since it creates dense clusters far away from the warehouse. The store-count method shows that even cluster size is less relevant than balanced demand.

9.4 Cost Factors

The cost-sensitivity tests show that the model adapts to changes in the parameters. Charger investment cost has the largest impact on the plan, while driver waiting cost also matters. Changes in electricity prices do not impact the charging station placement decision, but rather influence the overall objective value.

9.5 Benefits of Store Charging

Charging at stores is a sensible option for electric trucks. Trucks already stop at stores to unload, so they can charge during their idle time. Store charging also reduces the need for energy grid reinforcment and expensive charging infrastructure at the warehouse.

The final result is a hypothetical plan ignoring real-world constraints such as grid capacity, land ownership, and local regulations. The model successfully identifies store locations with the highest value for charger placements.

9.6 Future Work

Several extensions could strengthen the results:

- Comparing clustering methods on the full 259-store instance to test whether the ranking observed at 96 stores still applies to the full scale experiment.

- Additional experiments with alternative vehicle types, charger portfolios, cost parameters, and demand scenarios.
- Changing the model to support different charger configurations at the warehouse.
- Allow Stage 3 more flexibility, for example by merging adjacent clusters, or removing routing restrictions for the second stage.
- Include operational factors such as time windows, driving delays and battery aging cost.

The thesis provides a complete pipeline for planning charging station placement. Extensions remain open, but the method has been shown to work for a large, real-world problem.

10 An Operational Tool for Retailers

The research focuses on a single retailer in Germany. To make the method usable beyond this case, a self-hostable web app has been developed. The application allows retailers to upload their historic delivery data, set the parameters to their own real values and obtain a charging station placement with robust upper bound costs. The complete code base is published with examples on GitHub under 'max-elia/cspp'. It can be used through the web dashboard but also over the command line. This chapter gives an overview of the dashboard's main components.

10.1 Motivation

Charging station planning is dynamic. Retailers need to update their plans as new stores open, energy prices change, or new charger and electric vehicle technology becomes available. The web application allows a planner to create a test case, run the optimization, and review the results through a graphical user interface.

10.2 Setup

Creating a new test case follows three steps:

1. **Source.** Select an existing data file or upload a new one.
2. **Filtering.** Choose which stores to include based on their distance from the warehouse.
3. **Clustering.** Split the stores into groups. The application provides a live preview on a map so that planners can compare different group sizes and clustering methods before starting the calculation.

10.3 Input Data

The input format is kept simple to make it easy to prepare it from standard logistics records. The application requires a single JSON instance file. This file contains the warehouse and store locations, and historical demand scenarios.

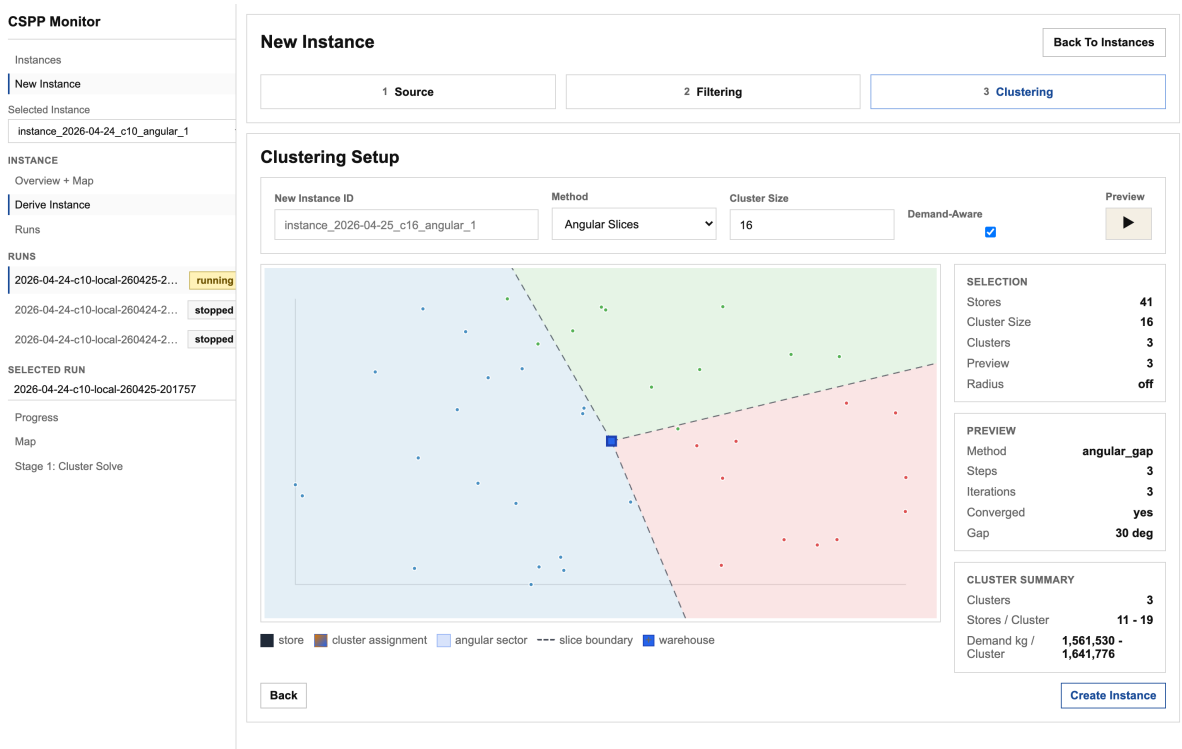

Figure 10.1: Planners can preview different clustering methods on the map and see summary statistics before starting the calculation.

The required top-level fields are:

- `schema_version`: the current schema version, set to `1`.
- `warehouse`: the depot location, with `latitude` and `longitude`.
- `stores`: the list of delivery locations.
- `demand_rows`: historic demand observations.

Each store entry needs a positive integer `client_num`, which acts as the solver node id, and its `latitude` and `longitude`. Additional fields such as `store_id`, `store_name`, address fields, are useful for identifying them in the map views of the web application but are not required for solving. Cluster assignments are optional. A retailer can either provide a `cluster_id` for every store or provide no clustering information at all. In the latter case, the application creates the clusters during the setup step; no `clustering_method` field is required in the input file. The demand table uses one row per store and delivery day: each row contains a `delivery_date`, a `client_num` that is the id of the store, and a non-negative `demand_kg`. ISO date strings such as `2026-01-15` are recommended. A minimal payload has the following structure:

```
{
  "schema_version": 1,
  "instance_id": "retailer-test",
  "warehouse": {
    "latitude": 48.0000,
    "longitude": 11.3000
  },
  "stores": [
    {
      "client_num": 1,
      "store_id": "STORE-001",
      "latitude": 48.1372,
      "longitude": 11.5761
    }
  ],
  "demand_rows": [
    {
      "delivery_date": "2026-01-15",
      "client_num": 1,
      "store_id": "STORE-001",
      "demand_kg": 1250.5
    }
  ]
}
```


The most important consideration for a retailer should be how to choose his demand scenarios. For robust planning, this should not only be an average week. It is useful to include days that were hard to serve in the past, especially days with high total demand, many active stores, broad geographic distribution, or other known operational difficulties. The resulting scenario set should still be small enough to solve, but diverse enough to represent the patterns against which the charging station placement should be protected.

Before running the optimization, the retailer can also adapt the planning parameters to its own setting. This includes selecting the vehicle type from a predefined set or specifying its own vehicle parameters, like available cargo load, battery capacity, available charger types and charger investment costs. Other parameters include electricity price, waiting cost, truck fixed cost per day, and available cores on the specific runtime. These parameters determine whether store-side charging is economically attractive in the specific use case; the uploaded demand and location data only define the physical delivery network.

10.4 Running the Solver and Tracking Progress

Once an instance is created, the user can set the parameters and start the optimization. The backend launches the run, persists the results, and monitors background jobs. The webapp supports multiple backends so the computing load can be offloaded to a virtual machine, keeping the heavy computational load off the users machine and enabling multiple parallel runs for comparing different settings. These compute targets are exposed as runtimes. Each runtime can also queue multiple runs. For SSH runtimes, the prepared run directory is synced to the remote project folder, and the full pipeline is started there. Intermediate state and final results are polled at regular intervals, and synced to the local machine.

While the calculation is in progress, the frontend progress panel shows the current stage, the elapsed time, and an estimate of the remaining time. The estimate is computed from the instance metadata and the configured number of processor cores. The application also provides real-time charts for each cluster, showing the best objective value over time.

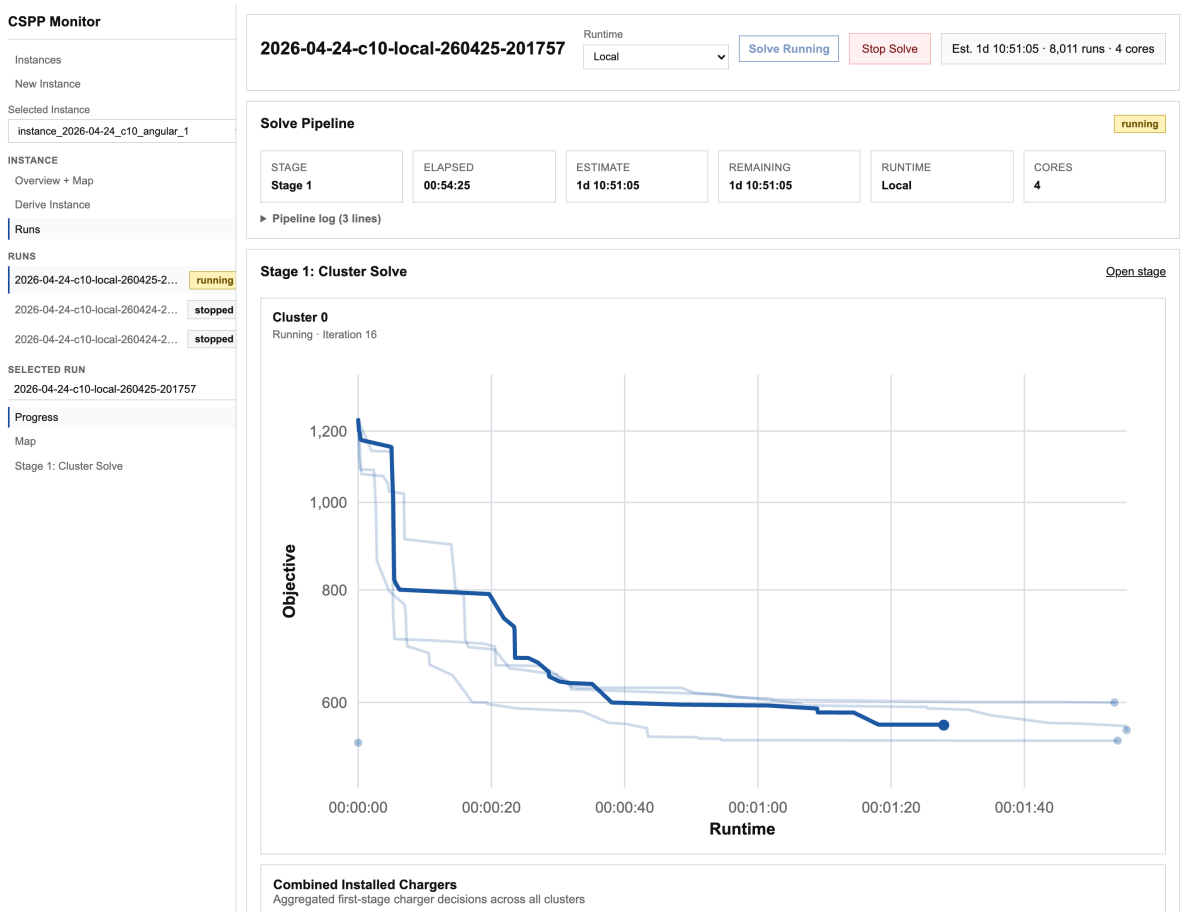

Figure 10.2: Progress view while the model is running.

10.5 Reviewing results

When the calculation is complete, the results are displayed on an interactive map. The user can inspect the chosen charger locations and the planned delivery routes for different scenarios. Detailed runtime charts for each cluster are also provided.

10.6 Applicability Beyond the Case Study

The application is designed to be flexible. Cost parameters can be adjusted to reflect the conditions of a different retailer. The contribution of this part of the thesis is a tool through which the same planning logic can be made generally available to other delivery networks.

11 Conclusion and Outlook

11.1 Conclusion

This thesis develops and tests a process for planning a retailer's long-term charging infrastructure from its own delivery data. The process identifies demand patterns, groups stores into clusters, chooses charger locations that are robust against the most difficult scenarios.

The main experiment uses 24 stores per cluster. It produces a plan with 95 chargers and a daily worst-case operating cost of EUR 17 378.72. Larger clusters lead to better plans, but also need more computation.

The clustering comparison shows that demand-aware angular slices give the lowest worst-case cost after reoptimization. The cost-sensitivity study shows that the model adapts well to changes of cost parameters. The web application packages the full pipeline so that retailers can apply the method to their own dataset.

11.2 Answers to Research Questions

1. **How can historical retailer delivery data be transformed into a two-stage robust CSPP whose first-stage charger decision optimizes for the worst observed demand days?** The thesis develops a pipeline that turns raw delivery records into optimization-ready instances. The two-stage robust model fixes the charging station placement in the first stage, and plans the operations for the realized demand of each day in the second stage.

2. **Which parameters drive the worst-case planning result, and how can realistic values be derived from vehicle specifications, empirical data, and planning assumptions?** Technical parameters are taken from public vehicle and charger specifications, operational values are derived from the retailer's records and assumptions are made for other parameters. The sensitivity study shows that charger investment cost has the strongest influence on the resulting plan, followed by driver waiting cost.

3. **Which clustering and runtime choices make the problem practically solvable without losing the economic meaning of the first-stage charger decision?** The full robust CSPP is too large for a single solve, so clustering, ASBP scenario addition, time-limited

solving, and reoptimization are combined into a three-stage pipeline. The experiments show that this gives useful results for the full 259-store instance, and that cluster size is a first-order design decision that affects both runtime and the resulting plan.

11.3 Future Research

The directions for further work are discussed in Section 9.6. The most promising extension is to update the charger portfolio as very fast heavy-duty charging matures. By 2026, Megawatt Charging System (MCS) had moved from a prototype topic to early commercial use: MAN already offers MCS equipment with production planned from the second quarter of 2026, while charger suppliers advertise modular systems up to about 1.2 MW [30, 31]. Future models should therefore test whether a small number of megawatt-capable warhouse chargers, or selected store chargers can replace part of the distributed lower-power store network. Further extensions include adding additional flexibility in how stores are assigned to clusters, and the inclusion of operational details such as traffic delays, time windows, and battery aging cost. The thesis provides a foundation on which these extensions can be built.

List of Figures

2.1 Unloading-time regression on 6,198 cleaned delivery stops 6

6.1 Stage 3 active-set reoptimization loop. The candidate plan is accepted only if it does not increase the total cost. If the plan is rejected but the worst scenario has changed, the loop continues with a new scenario in A. 37

7.1 Stage 1 objective trajectory 40
7.2 Stage 2 objective trajectory. 41

8.1 Cluster-size distribution in the 96-store clustering comparison. Store-count angular slices produce exactly balanced clusters, but this does not produce the best robust result. 46
8.2 Number of installed store chargers as different costs are changed. 48
8.3 Full 259-store instance. Size shows highest daily demand. 51
8.4 Stage 3 reoptimization trace for the 12 stores per cluster run. Iteration 3 increases the objective value and is rejected, Iteration 2 is selected. 52
8.5 Clustering for the main experiment (11 clusters). 54
8.6 Installed store chargers in the main experiment. 55
8.7 Installed charger types in both full-instance runs. Total counts are nearly identical (94 vs. 95), but the smaller cluster run favors low-power chargers. . . 57

10.1 Instance builder, clustering step 62
10.2 Solver progress view, running state 64

List of Tables

4.3 Second-stage constraint blocks and what they enforce. 18

5.1 Main physical parameters used in the computational study. 26
5.2 Baseline economic parameters. 26
5.3 Store charger types used in the final experiments. 27
5.4 Fixed depot parameters. 27

7.1 Number of MIP solves for the two full-instance runs. Stage 1 reports the master iterations measured from the run logs (the 24-store run additionally used 18 continuation rounds across its clusters); Stage 2 is the planned $C \cdot S$ count; Stage 3 is the planned $I_3 \cdot (C + 2CS)$ count. The Stage 3 count for the 24-store run is only the first reoptimization attempt, after which the algorithm stops because no improvement is needed. 42
7.2 Shared runtime bucket definition. 43
7.3 Time limits used by the bucket policy. 43

8.1 96-store clustering comparison. The columns "Avg. cost", "Stage 2 worst", and "Worst after reopt." are daily operating costs in EUR; "Worst after reopt." is the robust objective after Stage 3. 46
8.2 Setup for the full-scale experiments. 49
8.3 Comparison of the completed full-instance runs. Runtime is wall-clock time per stage. Costs are EUR per worst evaluated scenario. 56

Acronyms

CSPP Charging Station Placement Problem. 2–4, 9, 12, 18, 23, 29, 30, 39, 66

MCS Megawatt Charging System. 67

MIP Mixed-Integer Programming. 29, 30, 33–35, 39, 41, 42, 49, 69

SoC State of Charge. 5, 6, 18, 26

Bibliography

[1] Statistisches Bundesamt (Destatis). *Straßengüterverkehr deutscher Lastkraftfahrzeuge nach Güterabteilungen*. [Online; Accessed: 28. Mar. 2026]. 2026. URL: https://www.destatis.de/DE/Themen/Branchen-Unternehmen/Transport-Verkehr/Gueterverkehr/Tabellen/lkw-gueterabteilungen.html (visited on 03/28/2026).

[2] Umweltbundesamt. *Energieverbrauch und Kraftstoffe*. [Online; Accessed: 28. Mar. 2026]. 2025. URL: https://www.umweltbundesamt.de/daten/verkehr/endenergieverbrauch-im-verkehr#energieverbrauch-nach-verkehrsbereichen-und-energietragern (visited on 03/28/2026).

[3] Umweltbundesamt. *Emissionen des Verkehrs*. [Online; Accessed: 28. Mar. 2026]. 2025. URL: https://www.umweltbundesamt.de/daten/verkehr/emissionen-des-verkehrs (visited on 03/28/2026).

[4] ADAC e.V. *Kraftstoffpreise auf neuem Jahrestief*. [Online; Accessed: 28. Mar. 2026]. 2025. URL: https://www.adac.de/news/kraftstoffpreise-12-maerz-2025/ (visited on 03/28/2026).

[5] ADAC e.V. *Mai bislang billigster Tankmonat des Jahres*. [Online; Accessed: 28. Mar. 2026]. 2025. URL: https://www.adac.de/news/kraftstoffpreise-monatsbilanz-mai-2025/ (visited on 03/28/2026).

[6] BDEW Bundesverband der Energie- und Wasserwirtschaft e.V. *BDEW-Strompreisanalyse Oktober 2025*. [Online; Accessed: 28. Mar. 2026]. 2025. URL: https://www.bdew.de/service/daten-und-grafiken/bdew-strompreisanalyse/ (visited on 03/28/2026).

[7] KfW. *Ladestationen für Elektrofahrzeuge – Unternehmen (Zuschuss Nr. 441)*. [Online; Accessed: 28. Mar. 2026]. 2025. URL: https://www.kfw.de/inlandsfoerderung/Unternehmen/Energie-und-Umwelt/F%C3%B6rderprodukte/Ladestationen-f%C3%BCr-Elektrofahrzeuge-Unternehmen-(441)/ (visited on 03/28/2026).

[8] Umweltbundesamt. *Vollzug 38. BImSchV: Anrechnung von Strom für Elektrofahrzeuge*. [Online; Accessed: 28. Mar. 2026]. 2025. URL: https://www.umweltbundesamt.de/themen/verkehr/kraftstoffe-antriebe/vollzug-38-bimschv-anrechnung-von-strom-fuer (visited on 03/28/2026).

[9] Toll Collect GmbH. *Verlängerung der Mautbefreiung für emissionsfreie Fahrzeuge und neue Einbuchungsmöglichkeit mit TollNow*. [Online; Accessed: 28. Mar. 2026]. 2025. URL: https://www.toll-collect.de/de/toll_collect/bezahlen/mautbefreiung/mautbefreiung.html (visited on 03/28/2026).

[10] Mercedes-Benz Trucks. *eActros 300/400*. [Online; Accessed: 23. Mar. 2026]. 2026. URL: https://www.mercedes-benz-trucks.com/hu/hu/trucks/eactros-300-400.html (visited on 03/23/2026).

[11] Volvo Trucks. *Volvo FM Electric*. [Online; Accessed: 23. Mar. 2026]. 2026. URL: https://www.volvotrucks.in/en-in/trucks/alternative-fuels/electric-trucks/volvo-fm-electric.html (visited on 03/23/2026).

[12] Volvo Trucks. *Our Electric Truck Range*. [Online; Accessed: 23. Mar. 2026]. 2026. URL: https://www.volvotrucks.com/en-en/trucks/electric.html (visited on 03/23/2026).

[13] Renault Trucks. *Renault Trucks E-Tech D and D Wide*. [Online; Accessed: 23. Mar. 2026]. 2026. URL: https://www.renault-trucks.com/en-en/trucks/electric/e-tech-d-and-d-wide.html (visited on 03/23/2026).

[14] MAN Truck & Bus. *MAN eTGL*. [Online; Accessed: 23. Mar. 2026]. 2026. URL: https://www.man.eu/hr/hr/kamion/svi-modeli/man-etgl/man-etgl.html (visited on 03/23/2026).

[15] Daimler Truck AG. *Mercedes-Benz eActros 300/400 Technical Data*. Technical Report. [Online; Accessed: 20. Nov. 2025]. 2024. URL: https://www.daimlertruck.com (visited on 11/20/2025).

[16] K. Skrucha and M. Mądziel. "Research of Load Impact on Energy Consumption in an Electric Delivery Vehicle Based on Real Driving Conditions". In: *Energies* 16.2 (2023). [Accessed: 20. Nov. 2025], p. 775. DOI: 10.3390/en16020775.

[17] B. Borlaug, M. Muratori, M. Gilleran, D. Woody, W. Muston, T. Canada, A. Ingram, H. Gresham, and C. McQueen. "Heavy-Duty Truck Electrification and the Impacts of Depot Charging on Electricity Distribution Systems". In: *Nature Energy* 6.6 (2021), pp. 673–682. DOI: 10.1038/s41560-021-00855-0.

[18] VDE FNN. *VDE FNN Impuls Netzanschlüsse für E-Lkw-Ladeparks: Orientierungshilfe für Verteilnetzbetreiber und Netzanschlussnehmer für die Bearbeitung von Netzanschlussanfragen für E-Lkw-Ladeinfrastruktur*. [Online; Accessed: 25. Mar. 2026]. 2024. URL: https://www.vde.com/resource/blob/2347520/3490f979067fbbfd344729fcb61057ea/vde-fnn-impuls-netzanschluesse-fuer-e-lkw-ladeparks-data.pdf (visited on 03/25/2026).

[19] G. Hiermann, J. Puchinger, S. Ropke, and R. F. Hartl. "The Electric Fleet Size and Mix Vehicle Routing Problem with Time Windows and Recharging Stations". In: *European Journal of Operational Research* 252.3 (2016), pp. 995–1018. DOI: 10.1016/j.ejor.2016.01.038.

[20] M. Schiffer, P. S. Klein, G. Laporte, and G. Walther. "Integrated planning for electric commercial vehicle fleets: A case study for retail mid-haul logistics networks". In: *European Journal of Operational Research* 291.3 (2021), pp. 944–960. DOI: 10.1016/j.ejor.2020.09.054.

[21] M. Goerigk, D. Henke, J. Kager, F. Schäfer, and C. Thielen. *A Fast Approximate Column-and-Constraint Generation Method for Two-Stage Robust Mixed-Integer Programs*. Preprint submitted to Discrete Applied Mathematics, dated 06. Oct. 2025. 2025.

[22] R. Rahmaniani, T. G. Crainic, M. Gendreau, and W. Rei. "The Benders Decomposition Algorithm: A Literature Review". In: *European Journal of Operational Research* 259.3 (2017), pp. 801–817. DOI: 10.1016/j.ejor.2016.12.005.

[23] M. L. Fisher and R. Jaikumar. "A Generalized Assignment Heuristic for Vehicle Routing". In: *Networks* 11.2 (1981), pp. 109–124. DOI: 10.1002/net.3230110205.

[24] C. Kerscher and S. Minner. "Decompose-Route-Improve Framework for Solving Large-Scale Vehicle Routing Problems with Time Windows". In: *Transportation Research Part E: Logistics and Transportation Review* 204 (2025), p. 104409. DOI: 10.1016/j.tre.2025.104409.

[25] S.-N. Johnn, A. Miniguano-Trujillo, Y. Zhu, A. Gupte, and J. Kalcsics. *Stochastic Programming for an Integrated Assignment, Routing, and Scheduling Problem*. Optimization Online preprint. [Online; published 09. Feb. 2023, updated 29. Apr. 2024]. 2023. URL: https://optimization-online.org/2023/02/stochastic-programming-for-an-integrated-assignment-routing-and-scheduling-problem/ (visited on 03/19/2026).

[26] J. Schmalstieg, S. Käbitz, M. Ecker, and D. U. Sauer. "A holistic aging model for Li(NiMnCo)O2 based 18650 lithium-ion batteries". In: *Journal of Power Sources* 257 (2014), pp. 325–334.

[27] C. Cataldo-Díaz, R. Linfati, and J. W. Escobar. "Mathematical Model for the Electric Vehicle Routing Problem Considering the State of Charge of the Batteries". In: *Sustainability* 14.3 (2022), p. 1645. DOI: 10.3390/su14031645.

[28] Bundesministerium der Finanzen. *AfA-Tabelle für den Wirtschaftszweig "Personen- und Güterbeförderung (im Straßen- und Schienenverkehr)"*. [Online; Accessed: 23. Mar. 2026]. 1997. URL: https://www.bundesfinanzministerium.de/Content/DE/Standardartikel/

Themen/Steuern/Weitere_Steuerthemen/Betriebspruefung/AfA-Tabellen/AfA-Tabelle_Personen-und-Gueterbefoerderung.html (visited on 03/23/2026).

[29] DKE, VDE FNN, BDEW, ZVEH, ZVEI, and VDA. *Technischer Leitfaden Ladeinfrastruktur Elektromobilität, Version 4.1*. [Online; Accessed: 25. Mar. 2026]. 2023. URL: https://www.dke.de/resource/blob/988408/3c3f33c4cd7be00c569748c648745b47/technischer-leitfaden-ladeinfrastruktur-elektromobilitaet---version-4-data.pdf (visited on 03/25/2026).

[30] MAN Truck & Bus. *MAN demonstrates megawatt charging in Swedish winter*. [Online; Accessed: 29. Apr. 2026]. 2026. URL: https://press.mantruckandbus.com/corporate/man-demonstrates-megawatt-charging-in-swedish-winter/ (visited on 04/29/2026).

[31] Kempower. *Kempower Megawatt Charging System*. [Online; Accessed: 29. Apr. 2026]. 2026. URL: https://kempower.com/kempower-megawatt-charging-system/ (visited on 04/29/2026).

Declaration of AI Use

The following AI tools were used to support me in the development of my thesis: Gemini, Claude, and OpenAI's Codex. They were mainly used for the following tasks:

- **Idea exploration and clarification:** In the research phase to clarify background concepts relevant to the thesis such as basic physics of batteries, energy consumption in electric vehicles, and questions I had for parameter calibration.
- **Programming and mathematical support:** to better understand how mixed-integer programming constraints are written formally and how they can be translated into Python code, to better understand the python code of the adapted ASBP implementation and for general code completion, debugging, and boilerplate generation. All generated code was checked, tested, and adapted by me. The core algorithm design, model structure, and implementation decisions are my own work.
- **Language support:** Improved grammar and sentence structure in texts that I had written myself. I checked all suggested phrasing and adapted it so that it matched my own taste and intentions.

I followed the guidelines of the Professorship of Optimization and Sustainable Decision Making by manually verifying all AI-generated suggestions before using them. The robust CSPP formulation, the experimental design, the implementation decisions, and the analysis of the results were developed by me. No AI-generated text was included verbatim in the final thesis without manual revision. I take full responsibility for the originality and correctness of the submitted work.